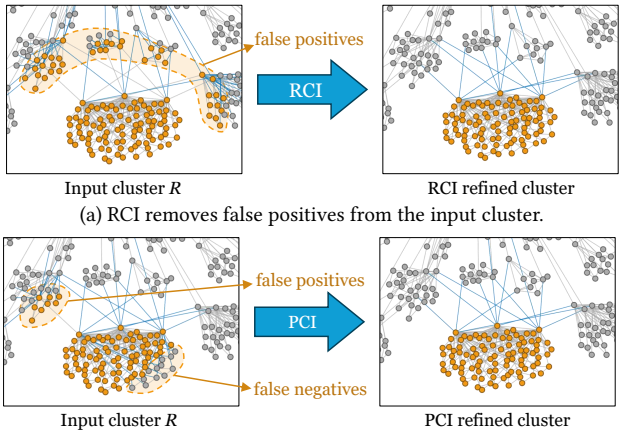
Strongly Local Convex-Programming Algorithms for Cluster Refinement on Large Graphs

ABSTRACT

Cluster refinement improves candidate clusters produced by upstream procedures by strengthening their internal connectivity and external separation, supporting diverse applications in graph analytics. We study two conductance-based refinement problems: restricted conductance improvement (RCI) and penalized conductance improvement (PCI). Existing exact algorithms repeatedly solve max-flow problems on local networks, making refinement queries expensive for large reference sets. To address this bottleneck, we derive convex formulations for both problems and develop lightweight Frank–Wolfe (FW) algorithms. We establish convergence guarantees for the FW iterations and prove that both algorithms are strongly local, with time and space complexities bounded in terms of R and fixed parameters, independently of the global graph size. Furthermore, we use the high-quality solutions returned by our FW algorithms to warm-start existing flow-based algorithms, yielding the exact result while achieving higher efficiency. Extensive experiments on ten real-world graphs and case studies demonstrate substantial speedups, high approximation quality, and practical utility.



(a) RCI removes false positives from the input cluster.

(b) PCI removes false positives and recovers false negatives.

Figure 1: Examples of RCI and PCI cluster refinement.

1 INTRODUCTION

In graph analytics, a *cluster* is commonly understood as a local subgraph that is well connected internally but sparsely connected to the rest of the graph [33, 53]. To capture clusters on real-world graphs, practical systems often first obtain candidate vertex sets from upstream procedures and then refine these sets for use by downstream applications and models [11, 15, 17, 24, 36, 45]. Such refinement may involve trimming or augmentation to strengthen the internal cohesion and external separation of the candidate sets [9, 15, 49, 52]. We refer to this task as a *cluster refinement* query: given a graph G and a reference set R, it returns a refined set S with improved cluster structure. Candidate sets can originate from a wide range of upstream procedures: local clustering algorithms such as personalized PageRank and other graph diffusions [2, 22, 43], k-hop neighborhood expansions [1, 27, 50], and query-driven retrieval [10, 16, 21, 37, 46]. This broad applicability makes cluster refinement a useful primitive for applications such as community detection [7, 25, 36, 44, 47], graph partitioning [13, 18, 32], protein-complex discovery [30, 40], and image segmentation [15, 28, 39].

To guide cluster refinement, a common measure is *conductance* [33], which compares the number of *cut edges* connecting a vertex set to the rest of the graph with its *volume*, defined as the sum of the degrees of its vertices. A low-conductance set therefore has few boundary edges (external connections) relative to its total degree mass (internal connections). Two well-studied conductance-based refinement problems are *restricted conductance improvement* (RCI) [5, 15, 26] and *penalized conductance improvement* (PCI) [15, 29, 41]. Given a candidate-set query R, RCI finds a subset $S \subseteq R$ that minimizes conductance. It can remove irrelevant vertices but cannot recover vertices outside R, making it suitable when R is expected to contain the target cluster. In contrast, PCI can both remove vertices from R and add vertices outside it, while penalizing the volume added outside R in its objective. PCI is therefore suitable for finding a target cluster near R through both trimming and augmentation. For example, Figure 1 illustrates RCI and PCI refining two candidate sets on a web graph[1]. In Figure 1a, the input set contains the entire target cluster along with irrelevant vertices (false positives); RCI trims the false positives and recovers the target cluster. In Figure 1b, the initial set misses some target-cluster vertices (false negatives) while including irrelevant vertices (false positives); PCI adds the missing vertices and removes the irrelevant ones, again recovering the target cluster exactly.

Existing Studies. The state-of-the-art algorithms for RCI and PCI are based on max-flow computation [26, 29, 41]. We refer to them as RCI-Flow and PCI-Flow, respectively. They use a Dinkelbach-style iteration that repeatedly solves max-flow/min-cut problems on local networks, updating an estimate of the optimal objective value until an exact solution is obtained. A key feature of both algorithms is *strong locality*: their running times depend on quantities associated with the input set R (such as $|R|$ and the volume of R) but not on the global graph size. This property is especially important for processing large real-world graphs, because it allows each query to access only a limited region around R rather than the entire graph. However, strong locality does not make the local computation cheap: each Dinkelbach round of RCI-Flow and PCI-Flow requires at least one exact max-flow computation. In particular, PCI-Flow employs a *expandable network* technique that repeatedly expands the local flow network to obtain a max-flow, further adding runtime and implementation complexity [15]. These repeated computations dominate query latency when the reference set is large or refinement queries are frequent.

Our Approaches. Motivated by these limitations, we develop lightweight convex-optimization algorithms that accelerate RCI and PCI without sacrificing strong locality. For RCI, we transform the problem into a local convex program, which can be solved by the standard Frank–Wolfe (FW) framework [20]. Specifically, each FW iteration requires only a single pass over the edges induced by R and performs simple edge-wise updates, taking time linear in the size of the subgraph induced by R. Because each iteration remains within R, the resulting RCI-FW algorithm is strongly local. For PCI,

[1] Data source: https://networkrepository.com/web-webbase-2001.php.

we observe that each Dinkelbach iteration does not necessarily require an exact max-flow solution to improve the current objective value. Instead, an approximate solution produced by lightweight FW iterations is sufficient to make progress. We therefore replace each expensive max-flow computation with FW iterations that perform only simple edge-wise updates. To avoid scanning the entire graph and preserve strong locality, we further observe that only a local region around R can influence these updates and develop an *active-set* technique that restricts computation to this region. The resulting PCI-FW algorithm processes only edges incident to a small set of active vertices. Furthermore, we prove that the active-set technique is not a heuristic approximation but is equivalent to the corresponding global FW updates.

Locality Guarantee. Strong locality based on locally expandable flow networks has been studied in prior work [9, 15, 19, 29, 41, 52]. However, to our best knowledge, no prior work has established strong locality for an active-set-based FW method by bounding the volume of its dynamically maintained active region. We therefore provide a nontrivial and rigorous strong-locality proof for PCI-FW. Specifically, we analyze how the edge-wise FW updates propagate beyond R and prove that the active-set volume remains bounded by quantities associated with R, independently of $|V|$ and $|E|$. Consequently, both the time and space complexities of PCI-FW are strongly local. This analysis is a vital theoretical contribution of our work and provides a formal guarantee for the active-set technique.

Optimality Guarantee. To combine the efficiency of FW with exact optimality guarantees, we use the approximate solutions returned by RCI-FW and PCI-FW to warm-start the existing flow-based algorithms. This hybrid strategy yields two exact algorithms, RCI-FW-Exact and PCI-FW-Exact. Each algorithm uses the objective value returned by its FW-based counterpart to warm-start the exact max-flow-based Dinkelbach procedure. Because the FW solution is typically already optimal or close to optimal, this warm start substantially reduces the number of expensive max-flow computations while preserving exactness.

Contributions. We summarize our contributions as follows:

- We develop convex formulations for RCI and PCI and design two efficient FW-based algorithms, RCI-FW and PCI-FW, that compute high-quality approximate solutions without invoking max-flow.
- We establish convergence guarantees for the FW procedures underlying both algorithms and prove that their computation is strongly local, with complexity bounds independent of the global graph size.
- We design two exact hybrid algorithms, RCI-FW-Exact and PCI-FW-Exact, combining the efficiency of FW with exact optimality guarantees.
- We conduct extensive experiments on ten real-world graphs and case studies, demonstrating the efficiency, effectiveness, and practical utility of our algorithms.

Reproducibility and full version paper. The code of our proposed algorithms and the full version of this paper are available at https://anonymous.4open.science/r/convex-conductance.

2 PRELIMINARIES

In this paper, we consider an undirected, unweighted, and connected graph $G = (V, E)$. For a vertex $u \in V$, let $d_u = |\{v \mid (u, v) \in E\}|$ denote its degree. For a vertex set $S \subseteq V$, we use $\mathrm{vol}(S) = \sum_{u \in S} d_u$ to denote its volume, and $\mathrm{cut}(S) = |\{(u, v) \in E \mid u \in S, v \notin S\}|$ to denote the number of edges crossing from S to $V \setminus S$. Next we introduce *conductance* and the *restricted conductance improvement (RCI)* problem.

Definition 1. **(Conductance)** *For a non-empty vertex set $S \subseteq V$, its conductance is*

$$\phi(S) = \frac{\mathrm{cut}(S)}{\min\{\mathrm{vol}(S), \mathrm{vol}(V \setminus S)\}}.$$

Problem 1. **(Restricted Conductance Improvement, RCI)** [5, 15, 26] *Given G and a non-empty set $R \subseteq V$ with $\mathrm{vol}(R) \leq \mathrm{vol}(V)/2$, find a vertex set $S^* \subseteq R$ that minimizes the conductance:*

$$S^* \in \arg\min_{\emptyset \neq S \subseteq R} \phi(S) = \arg\min_{\emptyset \neq S \subseteq R} \frac{\mathrm{cut}(S)}{\mathrm{vol}(S)}.$$

Note that, because RCI assumes $\mathrm{vol}(R) \leq \mathrm{vol}(V)/2$, every feasible set $S \subseteq R$ satisfies $\mathrm{vol}(S) \leq \mathrm{vol}(V \setminus S)$, and thus the minimum operation $\min\{\mathrm{vol}(S), \mathrm{vol}(V \setminus S)\}$ in the denominator of conductance simplifies to the single term $\mathrm{vol}(S)$. This makes RCI fundamentally different from the classical global conductance minimization problem, which searches over all vertex sets $S \subseteq V$ for the set with minimum conductance. First, global conductance minimization is NP-hard and therefore has no known polynomial-time exact algorithm [53], whereas the denominator simplification in RCI makes it polynomial-time solvable. Second, RCI assumes that an upstream procedure has already produced a local reference set R. The locality of R makes the assumption $\mathrm{vol}(R) \leq \mathrm{vol}(V)/2$ natural in graph-query applications, so this simplification usually does not restrict practical use.

The shrink-only nature means that RCI can only remove irrelevant vertices and cannot add missing ones. We next introduce *penalized conductance* and the *penalized conductance improvement (PCI)* problem, which extend the refinement model by allowing additions outside R at a controlled penalty.

Definition 2. **(Penalized Conductance)** [5, 15, 29, 41] *Given a reference set $R \subseteq V$ and a penalty parameter $\theta \in \mathbb{R}^+$, define the penalized volume of $S \subseteq V$ as*

$$\mathrm{vol}_\theta(S; R) = \mathrm{vol}(S \cap R) - \theta\,\mathrm{vol}(S \setminus R).$$

The penalized conductance of S with respect to R is

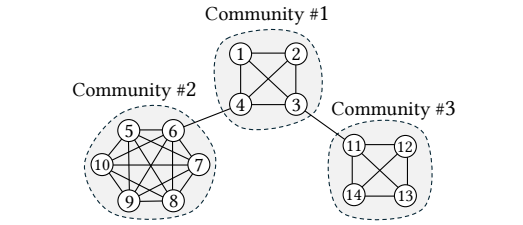
$$\phi_\theta(S; R) = \begin{cases} \dfrac{\mathrm{cut}(S)}{\mathrm{vol}_\theta(S; R)}, & \mathrm{vol}_\theta(S; R) > 0, \\ +\infty, & \mathrm{vol}_\theta(S; R) \leq 0. \end{cases}$$

Problem 2. **(Penalized Conductance Improvement, PCI)** [5, 15, 29, 41] *Given G, a non-empty set $R \subseteq V$, and a penalty parameter $\theta \in \mathbb{R}^+$, find a vertex set $S^* \subseteq V$ that minimizes the penalized conductance:*

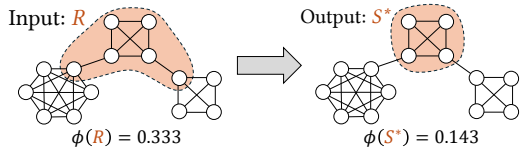
$$S^* \in \arg\min_{S \subseteq V} \phi_\theta(S; R).$$

Intuitively, PCI allows both deleting vertices from R and adding vertices outside R. Under the penalized conductance definition $\phi_\theta(S; R)$, vertices inside R contribute positive volume, while vertices outside R contribute negative volume scaled by the penalty parameter θ. The constraint $\mathrm{vol}_\theta(S; R) > 0$ prevents the solution from adding too many external vertices, and a larger θ makes expansion outside R more expensive. Note that when $\theta = +\infty$, adding any vertex outside R is prohibited, and PCI reduces to RCI. Therefore, PCI can be viewed as a generalization of RCI.

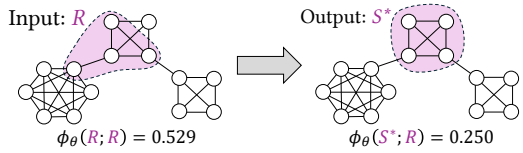
Example 1. *Figure 2a presents an example graph containing three tightly connected communities. For the RCI example in Figure 2b, let $R = \{1, 2, 3, 4, 6, 11\}$. This reference set contains all vertices of Community #1 together with two extraneous vertices, 6 and 11. The optimal RCI solution removes these two vertices and returns Community #1 $S^* = \{1, 2, 3, 4\}$. For the PCI example in Figure 2c, let $R = \{1, 3, 4, 6\}$ and $\theta = 1.0$. Here, R contains the extraneous vertex 6 but misses vertex*



(a) Example graph with three communities.



(b) Example of solving the RCI problem.



(c) Example of solving the PCI problem ($\theta = 1.0$).

Figure 2: Illustrative examples of the RCI and PCI problems.

2 *of Community #1. By both removing* 6 *and adding* 2*, the optimal PCI solution again recovers Community #1* $S^* = \{1, 2, 3, 4\}$.

Because RCI and PCI are defined relative to the reference set R, they naturally motivate algorithms that explore only the local region near R rather than the entire graph. We next formalize this locality requirement as *strongly local*.

Definition 3. **(Strongly Local)** [15] *An algorithm is strongly local with respect to a reference set* R *if its asymptotic time complexity can be bounded by quantities depending on* R*, such as* $|R|$ *and* $\mathrm{vol}(R)$*, but not depending on the global graph size, such as* $|V|$ *or* $|E|$.

Problem definition. This paper studies how to solve RCI and PCI efficiently and strongly locally. For RCI, the input is (G, R) and the target is a vertex set $S^* \in \arg\min_{\emptyset \neq S \subseteq R} \phi(S)$. For PCI, the input is (G, R, θ) and the target is a vertex set $S^* \in \arg\min_{S \subseteq V} \phi_\theta(S; R)$.

3 EXISTING ALGORITHMS

Existing state-of-the-art (SOTA) algorithms for RCI and PCI follow a similar structure: they use a Dinkelbach-style parametric search as the outer framework, and solve a parametric min-cut problem on a local flow network in each inner step. We next introduce the two corresponding algorithms, RCI-Flow and PCI-Flow.

3.1 RCI-Flow algorithm for RCI problem

The existing state-of-the-art algorithm RCI-Flow [26] (shown in Algorithm 1) uses the Dinkelbach framework to iteratively drive an estimate δ toward the optimum ϕ^*. It first initializes δ as $\phi(R)$ (line 1). In each iteration of the "while" loop, the algorithm invokes RCI-LocalMinCut, which uses a max-flow/min-cut computation to find a set $S \in \arg\min_{S \subseteq R} F_\delta(S)$, where $F_\delta(S) = \mathrm{cut}(S) - \delta\,\mathrm{vol}(S)$ (line 3). This subproblem determines whether the current estimate δ has reached the optimum ϕ^*. If $\phi(S) < \delta$, then δ is not optimal, and the update $\delta \leftarrow \phi(S)$ moves it closer to ϕ^* (line 4). Otherwise, $\phi(S) \geq \delta$, or equivalently, $F_\delta(S) \geq 0$. Since S minimizes

Algorithm 1: RCI-Flow(G, R) [26]

Input: Graph $G = (V, E)$, reference set R with $\mathrm{vol}(R) \leq \mathrm{vol}(V)/2$.
Output: A vertex set $S^* \in \arg\min_{\emptyset \neq S \subseteq R} \phi(S)$.

1 $S \leftarrow R$, $\delta \leftarrow \phi(S)$;
2 **while** *true* **do**
3 $S \leftarrow$ RCI-LocalMinCut(G, R, δ);
4 **if** $\phi(S) < \delta$ **then** $\delta \leftarrow \phi(S)$;
5 **else** break the while loop;
6 **return** S;
7 **Procedure** RCI-LocalMinCut(G, R, δ)
8 Initialize an empty network $(V' = R \cup \{s, t\}, E', c)$, where s is source, t is sink, and E' is the set of arcs with capacities c;
9 **forall** $(u, v) \in E, u \in R, v \in R$ **do**
10 Add arc (u, v) and (v, u) to E' with capacity 1;
11 **forall** $u \in R$ **do**
12 Add arc (s, u) to E' with capacity $c(s, u) = \delta d_u$;
13 Let $b_u \leftarrow |\{(u, v) \in E \mid v \notin R\}|$;
14 Add arc (u, t) to E' with capacity $c(u, t) = b_u$;
15 Compute a minimum s-t cut $(S, V' \setminus S)$ in network (V', E', c);
16 **return** $S \setminus \{s\}$;

F_δ, for every nonempty $S' \subseteq R$, we have $F_\delta(S') \geq F_\delta(S) \geq 0 \Rightarrow \mathrm{cut}(S') - \delta\,\mathrm{vol}(S') \geq 0 \Rightarrow \phi(S') \geq \delta$. Thus, no feasible subset has conductance smaller than δ, implying $\delta = \phi^*$. The algorithm then terminates the loop and returns the current optimal solution (lines 5–6).

Theorem 1. [26] RCI-Flow *has a strongly local time complexity of* $O(\mathrm{cut}(R) \cdot T_{\mathrm{MC}}(|R|, \mathrm{vol}(R)))$*, where* $T_{\mathrm{MC}}(|R|, \mathrm{vol}(R))$ *denotes the time for solving one min-cut instance on a network with* $O(|R|)$ *vertices and* $O(\mathrm{vol}(R))$ *edges.*

For example, if using Dinic's max-flow algorithm [12], we have $T_{\mathrm{MC}}(|R|, \mathrm{vol}(R)) = O(|R|^2\,\mathrm{vol}(R))$, and thus RCI-Flow runs in $O(\mathrm{cut}(R)|R|^2\,\mathrm{vol}(R))$ time. Although this bound is strongly local, the algorithm still needs to solve a sequence of exact min-cut problems, which can be expensive when $|R|$ or $\mathrm{vol}(R)$ is large.

3.2 PCI-Flow algorithm for PCI problem

The existing SOTA algorithm PCI-Flow (shown in Algorithm 2) follows the same Dinkelbach framework as RCI-Flow [15, 29, 41]. Lines 1–6 implement the outer Dinkelbach iteration: each round uses procedure PCI-LocalMinCut to obtain a set $S \in \arg\min_{S \subseteq V} F_{\delta,\theta}(S)$, where $F_{\delta,\theta}(S) = \mathrm{cut}(S) - \delta\,\mathrm{vol}_\theta(S; R)$ (line 3). The algorithm then determines whether to continue the Dinkelbach iteration or terminate according to whether $\phi_\theta(S; R) < \delta$ (lines 4–6). To ensure strong locality, the procedure PCI-LocalMinCut computes the minimum cut using a locally expandable flow network. Specifically, it maintains an active vertex set A, initialized as R, and constructs the current flow network only around A. After each local max-flow computation, it expands A through saturated boundary edges. By repeatedly performing this expansion, the procedure eventually obtains the desired minimum cut from the local flow network. Therefore, PCI-Flow has the following strongly-local time complexity.

Theorem 2. [15, 29, 41] *Let* $\gamma = 1 + 1/\theta$. PCI-Flow *has a strongly local time complexity of* $O(\mathrm{cut}(R) \cdot T_{\mathrm{PCI-MC}})$*, where* $T_{\mathrm{PCI-MC}} = O(\gamma^2\,\mathrm{vol}(R)^2 \log(\gamma\,\mathrm{vol}(R)))$ *is the time of one invocation of* PCI-LocalMinCut *using the Dinic's blocking-flow framework* [41].

Although this complexity is strongly local, PCI-Flow still needs to solve a sequence of exact flow subproblems. Moreover, each flow

Algorithm 2: PCI-Flow(G, R, θ) [29, 41]

Input: Graph $G = (V, E)$, reference set R, and penalty parameter $\theta > 0$.
Output: A vertex set $S^* \in \arg\min_{S \subseteq V} \phi_\theta(S; R)$.

$S \leftarrow R$, $\delta \leftarrow \phi_\theta(S; R)$;
while *true* **do**
  $S \leftarrow$ PCI-LocalMinCut(G, R, θ, δ);
  if $\phi_\theta(S; R) < \delta$ **then** $\delta \leftarrow \phi_\theta(S; R)$;
  else break the while loop;
return S;
Procedure PCI-LocalMinCut(G, R, θ, δ)
  Let $A \leftarrow R$ be the active vertices set;
  Initialize an empty network $(V' = A \cup \{s, t\}, E', c)$, where s is source, t is sink, and E' is the set of arcs with capacities c;
  while *true* **do**
    $\partial A \leftarrow \{v \notin A \mid \exists u \in A, (u, v) \in E\}$;
    $V' \leftarrow V' \cup \partial A$;
    forall $u \in R$ **do**
      Add arc (s, u) with capacity $c(s, u) = \delta d_u$;
    forall $u \in (A \cup \partial A) \setminus R$ **do**
      Add arc (u, t) with capacity $c(u, t) = \delta\theta d_u$;
    forall *edge* (u, v) *incident to a vertex in* A **do**
      Add arcs (u, v) and (v, u) with capacity 1;
    Compute the maximum flow and a minimum s-t cut $(S, V' \setminus S)$ in the network;
    $\Delta A \leftarrow \{u \in \partial A \mid (u, t)$ is saturated$\}$;
    if $\Delta A = \emptyset$ **then return** $S \setminus \{s\}$;
    $A \leftarrow A \cup \Delta A$;

subproblem may require multiple expansions of the local flow network before it can certify a global minimum cut. These repeated flow computations and expansion steps are not only time-consuming, but also make the algorithm more complex to implement [15]. This motivates the convex-programming approach developed in the following sections.

4 NOVEL CONVEX-PROGRAMMING ALGORITHMS FOR RCI PROBLEM

To avoid the sequence of expensive maximum-flow computations required by RCI-Flow, we develop a convex-programming approach for RCI. We first follow a standard route by expressing RCI as a local linear program (LP) and deriving its edge-flow dual. Our key observation is that the dual's nonsmooth max–min objective can be equivalently replaced by a smooth quadratic objective. This nontrivial reformulation yields a convex program that can be efficiently optimized using the Frank–Wolfe (FW) algorithm. Finally, we combine the resulting FW solution with max-flow to obtain an efficient exact hybrid algorithm.

4.1 Local LP Formulation and Edge-Flow Dual

Local LP formulation RCI-LP**.** For each vertex $u \in R$, let $b_u = |\{(u, v) \in E \mid v \notin R\}|$ denote its *boundary degree* with respect to R, and let $E_R = \{(u, v) \in E \mid u, v \in R\}$ denote the set of edges internal to R. We propose the following local LP formulation.

$$\text{(RCI-LP)} \qquad \min_{x} \sum_{(u,v)\in E_R} |x_u - x_v| + \sum_{u \in R} b_u x_u$$
$$\text{s.t.} \quad \sum_{u \in R} d_u x_u = 1,$$
$$x_u \geq 0, \quad \forall u \in R.$$

Given a vertex set $S \subseteq R$, setting $x_u = 1/\mathrm{vol}(S)$ for every $u \in S$ and $x_u = 0$ otherwise yields an objective value of RCI-LP equal to $\phi(S)$. Therefore, the objective RCI-LP can be viewed as a continuous relaxation of conductance. The formulation is local because the influence of $V \setminus R$ is summarized by the boundary degrees b, while all variables are defined only over R. We next prove that solving RCI-LP is equivalent to solving the RCI problem: its optimal value equals the RCI optimum, and an optimal RCI set can be recovered from an optimal LP solution.

Theorem 3. *Let ϕ^* be the optimal RCI value, and* OPT(RCI-LP) *be the optimal objective value of* RCI-LP*. Then,* OPT(RCI-LP) $= \phi^*$*, and an optimal RCI set can be recovered by thresholding any optimal* RCI-LP *solution.*

Proof. "OPT(RCI-LP) $\leq \phi^*$": let S^* be an optimal RCI set, and define $x^*_u = 1/\mathrm{vol}(S^*)$ for $u \in S^*$ and $x^*_u = 0$ otherwise. It can be easily derived that x^* vector is feasible and the objective value is ϕ^*. Therefore, OPT(RCI-LP) $\leq \phi^*$.

"OPT(RCI-LP) $\geq \phi^*$": let x be any feasible solution to RCI-LP, and define the threshold set $S_t = \{u \in R : x_u > t\}$ for $t \geq 0$. The layer-cake and discrete coarea identities give

$$\int_0^\infty \mathrm{vol}(S_t)\, dt = \sum_{u \in R} d_u x_u = 1$$

and

$$\int_0^\infty \mathrm{cut}(S_t)\, dt = \sum_{(u,v)\in E_R} |x_u - x_v| + \sum_{u \in R} b_u x_u.$$

By the definition of ϕ^*, $\mathrm{cut}(S_t) \geq \phi^* \mathrm{vol}(S_t)$ for every $t \geq 0$. Integrating this inequality yields

$$\sum_{(u,v)\in E_R} |x_u - x_v| + \sum_{u \in R} b_u x_u \geq \phi^* \int_0^\infty \mathrm{vol}(S_t)\, dt = \phi^*.$$

Since this holds for every feasible x, OPT(RCI-LP) $\geq \phi^*$. Together with OPT(RCI-LP) $\leq \phi^*$, we have OPT(RCI-LP) $= \phi^*$.

Recover optimal set from RCI-LP: let $\bar{x}$ be an optimal RCI-LP solution and let $\bar{S}_t = \{u \in R : \bar{x}_u > t\}$. Using the two integral identities above and OPT(RCI-LP) $= \phi^*$, we obtain

$$\phi^* = \int_0^\infty \mathrm{cut}(\bar{S}_t)\, dt = \int_0^\infty \frac{\mathrm{cut}(\bar{S}_t)}{\mathrm{vol}(\bar{S}_t)} \mathrm{vol}(\bar{S}_t)\, dt.$$

Since $\int_0^\infty \mathrm{vol}(\bar{S}_t)\, dt = 1$, the right-hand side is a weighted average of the conductances of the nonempty threshold sets. Moreover, since $\mathrm{cut}(\bar{S}_t)/\mathrm{vol}(\bar{S}_t) \geq \phi^*$, every nonempty $\bar{S}_t$ satisfies $\phi(\bar{S}_t) = \phi^*$ and is therefore an optimal RCI solution. □

Edge-flow dual RCI-DP**.** We next derive the dual of RCI-LP. Following the standard LP duality procedure, for each internal edge $e = (u, v) \in E_R$, we introduce two endpoint edge-flow variables $f_{e,u}$ and $f_{e,v}$, deriving the dual program RCI-DP:

$$\text{(RCI-DP)} \qquad \max_{\eta, f} \quad \eta$$
$$\text{s.t.} \quad \frac{b_u + \sum_{e \in E_R : e \ni u} f_{e,u}}{d_u} \geq \eta, \quad \forall u \in R,$$
$$f_{e,u} + f_{e,v} = 0, \quad \forall e = (u, v) \in E_R,$$
$$|f_{e,u}| \leq 1, \quad \forall e = (u, v) \in E_R.$$

To interpret this dual, view every internal edge $e = (u, v)$ as a flow channel of capacity 1. The antisymmetry constraint $f_{e,u} + f_{e,v} = 0$ ensures that any load sent away from one endpoint is received by the other; for example, $f_{e,u} = -0.7$ and $f_{e,v} = 0.7$ represent a

transfer of 0.7 units from u to v. For a feasible edge flow vector f, define the *normalized flow load* at vertex u as

$$\rho_u(f) = \frac{b_u + \sum_{e \in E_R : e \ni u} f_{e,u}}{d_u}.$$

Given a feasible edge flow f, the first constraint implies that the largest feasible value of η is $\eta = \min_{u \in R} \rho_u(f)$. Therefore, the objective of maximizing η is equivalent to finding a feasible f that maximizes $\min_{u \in R} \rho_u(f)$, i.e.,

$$\mathrm{OPT}(\text{RCI-DP}) = \max_f \eta = \max_f \min_{u \in R} \rho_u(f).$$

4.2 Convex-Programming Algorithm RCI-FW

Exact quadratic formulation RCI-CP. The max-min objective in RCI-DP gives an exact dual characterization of RCI. However, this objective is nonsmooth and lacks strong convexity, making it difficult to optimize efficiently using first-order methods. In this subsection, we introduce a novel and key observation: the max-min objective can be equivalently replaced by the quadratic objective $\frac{1}{2}\sum_{u \in R} d_u \rho_u(f)^2$. Intuitively, this quadratic objective smooths the direct optimization of the minimum load into a global balancing problem, where the quadratic terms penalize vertices with large $\rho_u(f)$ values and thereby encourage the loads to become globally balanced. Based on this observation, we propose the following convex-programming formulation:

$$\begin{aligned} \text{(RCI-CP)} \qquad \min_f \quad & \frac{1}{2}\sum_{u \in R} d_u \rho_u(f)^2 \\ \text{s.t.} \quad & f_{e,u} + f_{e,v} = 0, \quad \forall e = (u,v) \in E_R, \\ & |f_{e,u}| \le 1, \quad \forall e = (u,v) \in E_R. \end{aligned}$$

We next prove that this smooth reformulation is exact for RCI.

Theorem 4. *Let f^* be an optimal solution to* RCI-CP*, and let $\rho_{\min} = \min_{u \in R} \rho_u(f^*)$. Then $\rho_{\min} = \phi^*$, and the set $S_{\min} = \{u \in R \mid \rho_u(f^*) = \rho_{\min}\}$ is an optimal RCI solution.*

Proof. We first show that every edge $e = (u,v) \in E_R$ with $u \in S_{\min}$ and $v \in R \setminus S_{\min}$ satisfies $f^*_{e,u} = 1$. By the definition of $S_{\min}$, we have $\rho_u(f^*) = \rho_{\min} < \rho_v(f^*)$. Assume for contradiction that $f^*_{e,u} < 1$. Let $\epsilon = \min\{\frac{1 - f^*_{e,u}}{2}, \frac{\rho_v(f^*) - \rho_u(f^*)}{1/d_u + 1/d_v}\} > 0$. Increasing $f^*_{e,u}$ by ϵ and decreasing $f^*_{e,v}$ by ϵ strictly decreases the quadratic objective by $\epsilon\big(\rho_v(f^*) - \rho_u(f^*)\big) - \frac{\epsilon^2}{2}\big(\frac{1}{d_u} + \frac{1}{d_v}\big) > 0$. This contradicts the optimality of f^*. Hence, every edge crossing from $S_{\min}$ to $R \setminus S_{\min}$ is saturated toward $S_{\min}$.

We next sum the flow loads over $S_{\min}$. The contributions of edges with both endpoints in $S_{\min}$ cancel by antisymmetry, while every saturated edge crossing from $S_{\min}$ to $R \setminus S_{\min}$ contributes one unit. Therefore, $\rho_{\min}\,\mathrm{vol}(S_{\min}) = \sum_{u \in S_{\min}} d_u \rho_u(f^*) = \sum_{u \in S_{\min}} (b_u + |\{(u,v) \in E_R \mid u \in S_{\min}, v \notin S_{\min}\}|) = \mathrm{cut}(S_{\min})$. It follows that $\rho_{\min} = \phi(S_{\min})$.

Finally, $(\rho_{\min}, f^*)$ is feasible for RCI-DP, so $\rho_{\min} \le \mathrm{OPT}(\text{RCI-DP}) = \phi^*$ by strong LP duality and the equivalence between RCI-LP and RCI. Since $S_{\min} \subseteq R$ is a feasible RCI set, we also have $\phi^* \le \phi(S_{\min}) = \rho_{\min}$. Thus, $\rho_{\min} = \phi^*$ and $\phi(S_{\min}) = \phi^*$, proving that $S_{\min}$ is an optimal RCI solution. □

Frank–Wolfe based algorithm RCI-FW. Let $Q(f) = \frac{1}{2}\sum_{u \in R} d_u \rho_u(f)^2$ denote the objective of RCI-CP. For an edge $e = (u,v) \in E_R$, the partial derivative of $Q(f)$ with respect to $f_{e,u}$ is $\frac{\partial Q(f)}{\partial f_{e,u}} = \rho_u(f) - \rho_v(f)$. For example, if $\rho_u(f) < \rho_v(f)$, increasing $f_{e,u}$ (and correspondingly decreasing $f_{e,v}$) gives a descent direction of $Q(f)$; conversely, if $\rho_u(f) > \rho_v(f)$, decreasing $f_{e,u}$ gives a descent direction. More intuitively, transferring flow load toward the endpoint with the smaller ρ value follows the gradient-descent direction. Based on this flow-transfer principle, we design the FW-based algorithm RCI-FW, as shown in Algorithm 3.

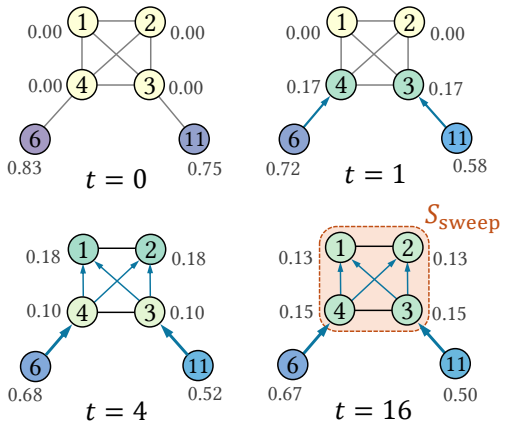
Algorithm 3: RCI-FW$(G, R, T, T_{\text{check}})$

Input: Graph $G = (V, E)$, reference set R, maximum number of iterations $T \in \mathbb{Z}_{>0}$, and checking interval $T_{\text{check}} \in \mathbb{Z}_{>0}$.
Output: A vertex set S_{best} approximating the optimal RCI solution.

1 **foreach** $e = (u,v) \in E_R$ **do** $f^{(0)}_{e,u} \leftarrow 0$, $f^{(0)}_{e,v} \leftarrow 0$;
2 **foreach** $u \in R$ **do** $\rho^{(0)}_u \leftarrow b_u / d_u$;
3 $S_{\text{best}} \leftarrow R$;
4 **for** $t = 1, 2, \ldots, T$ **do**
5 $\gamma^{(t)} \leftarrow \frac{2}{t+2}$;
6 **foreach** $e = (u,v) \in E_R$ **do**
7 **if** $\rho^{(t-1)}_u < \rho^{(t-1)}_v$ **then** $\hat{f}_{e,u} \leftarrow 1$, $\hat{f}_{e,v} \leftarrow -1$;
8 **else if** $\rho^{(t-1)}_u > \rho^{(t-1)}_v$ **then** $\hat{f}_{e,u} \leftarrow -1$, $\hat{f}_{e,v} \leftarrow 1$;
9 **else** $\hat{f}_{e,u} \leftarrow f^{(t-1)}_{e,u}$, $\hat{f}_{e,v} \leftarrow f^{(t-1)}_{e,v}$;
10 $f^{(t)}_{e,u} \leftarrow (1 - \gamma^{(t)}) f^{(t-1)}_{e,u} + \gamma^{(t)} \hat{f}_{e,u}$;
11 $f^{(t)}_{e,v} \leftarrow (1 - \gamma^{(t)}) f^{(t-1)}_{e,v} + \gamma^{(t)} \hat{f}_{e,v}$;
12 **foreach** $u \in R$ **do** $\rho^{(t)}_u \leftarrow \frac{b_u + \sum_{e \in E_R : e \ni u} f^{(t)}_{e,u}}{d_u}$;
13 **if** $t \bmod T_{\text{check}} = 0$ **then** // Periodically check, early-stop
14 Sort R as $u_1, u_2, \ldots, u_{|R|}$ in non-decreasing order of $\rho^{(t)}_u$;
15 $k^* \leftarrow \arg\min_{1 \le k \le |R|} \phi(\{u_1, u_2, \ldots, u_k\})$;
16 $S_{\text{sweep}} \leftarrow \{u_1, u_2, \ldots, u_{k^*}\}$;
17 **if** $\phi(S_{\text{sweep}}) < \phi(S_{\text{best}})$ **then** $S_{\text{best}} \leftarrow S_{\text{sweep}}$;
18 **else** Break the "for" loop;
19 **return** S_{best};

RCI-FW first initializes all edge flows to zero, yielding $\rho^{(0)}_u = b_u/d_u$ (lines 1–2), and initializes S_{best} to record the best solution found during the subsequent FW iterations (line 3). At each iteration, the algorithm uses the standard FW step size $2/(t+2)$ (line 5) and performs the FW update (lines 6–11). Specifically, it first sets the update direction $\hat{f}_{e,u}$ and $\hat{f}_{e,v}$ to send flow toward the endpoint with the smaller $\rho^{(t-1)}$ value (lines 7–8), and then computes the updated edge-flow variables $f^{(t)}$ (lines 9–10). In the special case where the two endpoints have equal $\rho^{(t-1)}$ (line 9), it sets $\hat{f}_{e,u} \leftarrow f^{(t-1)}_{e,u}$ and $\hat{f}_{e,v} \leftarrow f^{(t-1)}_{e,v}$, so the updates in lines 10–11 give $f^{(t)} \leftarrow f^{(t-1)}$, i.e., leave the flow unchanged. Based on the updated flow, the algorithm computes the updated flow loads $\rho^{(t)}$ (line 12). In addition to the fixed iteration limit T (line 4), we propose a sweep-cut-based early-stopping strategy (lines 13–18), which triggers a sweep-cut check every T_{check} iterations (line 13). During each check, the algorithm sorts the vertices in non-decreasing order of $\rho^{(t)}$ and selects the prefix set with the smallest conductance as the candidate set S_{sweep} (lines 14–16). If S_{sweep} improves upon the best solution S_{best} found so far, the algorithm updates the recorded solution (lines 17–18). Otherwise, the solution has not improved over the last T_{check} iterations, and the algorithm terminates the iteration (line 18). Finally, it returns the best recorded set S_{best} (line 19).

Example 2. *Figure 3 illustrates the execution of* RCI-FW *on the graph and reference set $R = \{1, 2, 3, 4, 6, 11\}$ from Figure 2b. Since* RCI-FW *performs all computations within R, only the subgraph over R is shown. Initially, S_{best} is set to R, with conductance $\phi(R) = 8/23$. At $t = 0$, only vertices 6 and 11 have positive initial loads, while the other vertices have $\rho^{(0)}_u = 0$. At $t = 1$, the flows on edges $(6, 4)$ and*



The grey number beside each vertex is its normalized flow load $\rho_u^{(t)}$, and the blue arrows indicate flow transfers.

Figure 3: A running example of RCI-FW.

$(11, 3)$ *are updated, transferring part of these loads to vertices* 4 *and* 3*, respectively; the blue arrows indicate the corresponding flow directions. By* $t = 4$*, the flow has propagated further through the internal edges, and the loads begin to balance among vertices* $\{1, 2, 3, 4\}$*. At* $t = 16$*, the loads of vertices* $\{1, 2, 3, 4\}$ *are already close to one another. A sweep cut at this iteration returns the minimum-conductance prefix* $S_{\text{sweep}} = \{1, 2, 3, 4\}$*, whose conductance is* $\phi(S_{\text{sweep}}) = 1/7$*. Since this improves upon the current best solution,* RCI-FW *updates* $S_{\text{best}} \leftarrow S_{\text{sweep}}$*, which exactly matches the optimal RCI solution. At the next sweep check, no prefix set improves upon the conductance of the current* S_{best}*, so the algorithm terminates and returns* S_{best}*.*

Next, in Theorem 5, we establish the approximation guarantee of the Frank–Wolfe procedure in RCI-FW.

Theorem 5. *For the Frank–Wolfe iterations in* RCI-FW*, let* $d_{\min} = \min_{u \in R} d_u$ *be the minimum degree in* R*. Then we have*

$$0 \le \phi^* - \rho_{\min}^{(t)} \le 4\sqrt{\frac{\text{vol}(R)}{d_{\min} t}}.$$

Let $S_{\text{sweep}}^{(t)}$ *be the subset obtained by performing the sweep cut in lines 13–15 at iteration* t*. Then,*

$$0 \le \phi(S_{\text{sweep}}^{(t)}) - \phi^* \le 2\sqrt{\frac{2\,\text{vol}(R)}{d_{\min}}} t^{-1/4}.$$

Proof. The detailed proof is presented in the appendix. □

As $t \to \infty$, Theorem 5 establishes that, on the one hand, $\min_{u \in R} \rho_u^{(t)}$ converges to ϕ^* at a rate of $O(1/\sqrt{t})$ and, on the other hand, the sweep-cut set S_{sweep} converges to an optimal RCI solution at a rate of $O(1/t^{1/4})$.

Complexities. Let $\tau \le T$ denote the number of Frank–Wolfe iterations performed before termination. Each iteration scans the internal edges E_R once, requiring $O(|R| + |E_R|) = O(\text{vol}(R))$ time. Each sweep-cut check sorts the vertices and evaluates all prefix sets in $O(|R| \log |R| + \text{vol}(R))$ time. Since a check is performed every T_{check} iterations, RCI-FW has a local time complexity of $O\left(\tau\,\text{vol}(R) + \lceil \frac{\tau}{T_{\text{check}}} \rceil |R| \log |R|\right)$, and a local space complexity of $O(|R| + |E_R|) = O(\text{vol}(R))$ space, making RCI-FW strongly local. As demonstrated in our experiments, its lightweight iterations and sweep-based early stopping enable RCI-FW to run efficiently in practice while returning solutions with conductance very close to the exact RCI optimum.

4.3 Exact Hybrid Algorithm RCI-FW-Exact

To obtain an exact solution while retaining the practical efficiency of convex programming, we combine RCI-FW with the flow-based exact algorithm RCI-Flow. We refer to this hybrid strategy as RCI-FW-Exact. Specifically, RCI-FW-Exact first runs RCI-FW to obtain S_{best} and then uses its conductance $\phi(S_{\text{best}})$ to warm-start the Dinkelbach iteration of RCI-Flow. Operationally, this only requires replacing line 1 of Algorithm 1 with $S \leftarrow S_{\text{best}}$ and $\delta \leftarrow \phi(S_{\text{best}})$; all subsequent Dinkelbach and minimum-cut steps remain unchanged.

This warm start does not affect exactness, since the conductance of S_{best} is a valid upper bound on the optimal RCI value. In practice, RCI-FW returns solutions very close to optimal, so the warm-started Dinkelbach procedure typically requires only about one max-flow computation. In contrast, directly running the Dinkelbach procedure in RCI-Flow usually requires about 8–12 iterations (see Exp-2 and Exp-4 in Section 6). This reduction in max-flow computations leads to a substantial efficiency improvement.

5 NOVEL CONVEX-PROGRAMMING ALGORITHMS FOR PCI PROBLEM

Compared with RCI, PCI is more expressive because it allows the solution to both shrink and expand the reference set R, but this additional flexibility makes algorithm design more challenging, especially when strong locality must be maintained. In this section, we develop our convex-programming solution for PCI. We first convert the fixed-δ subproblem to a convex-programming formulation and then present the Frank–Wolfe-based algorithm PCI-FW. Next, we establish its strong locality through maintaining a local active region. Finally, we combine PCI-FW with the flow-based algorithm PCI-Flow to obtain the exact algorithm PCI-FW-Exact.

5.1 Convex Formulations for Fixed-δ Subproblem

Recall that, in each Dinkelbach iteration, PCI-Flow uses a min-cut computation to exactly solve the fixed-δ subproblem $S \in \arg\min_{S \subseteq V} F_{\delta,\theta}(S)$, which is computationally expensive. We make the key observation that an exact minimizer of $F_{\delta,\theta}$ is not necessary for making progress: any set $\widetilde{S}$ satisfying $F_{\delta,\theta}(\widetilde{S}) < 0$ is sufficient. We call such a set a *negative witness*. This negative witness satisfies $F_{\delta,\theta}(\widetilde{S}) < 0 \Leftrightarrow \phi_\theta(\widetilde{S}; R) < \delta$, allowing a valid Dinkelbach update $\delta \leftarrow \phi_\theta(\widetilde{S}; R)$ that moves the current estimate toward the optimum. Consequently, we can replace the expensive max-flow computation with a cheaper search for a negative witness in each Dinkelbach iteration. To this end, we employ convex optimization, which is well suited to rapidly finding high-quality approximate solutions. We next formulate the minimization of $F_{\delta,\theta}$ as an LP, derive its dual program, and finally transform the dual into a smooth convex program.

Fixed-δ LP formulation PCI-LP**.** To formulate the negative-witness search as a continuous optimization problem, define the *penalized-volume weight* of every vertex $u \in V$ as

$$\omega_{\theta,u} = \begin{cases} d_u, & u \in R, \\ -\theta d_u, & u \notin R. \end{cases}$$

With this notation, the penalized volume can be written as the vertex-wise sum $\text{vol}_\theta(S; R) = \sum_{u \in S} \omega_{\theta,u}$, and hence $F_{\delta,\theta}(S) = \text{cut}(S) - \sum_{u \in S} \delta\omega_{\theta,u}$. Thus, S is a negative witness if and only if $\sum_{u \in S} \delta\omega_{\theta,u} - \text{cut}(S) > 0$. By replacing the discrete vertex set S with a continuous indicator vector x, we obtain the following LP

formulation:

$$\text{(PCI-LP)} \qquad \max_{x} \quad \sum_{u\in V} \delta\omega_{\theta,u} x_u - \sum_{(u,v)\in E} |x_u - x_v| \\ \text{s.t.} \quad 0 \le x_u \le 1, \quad \forall u \in V.$$

We next prove that PCI-LP is an exact formulation of the fixed-δ negative-witness problem.

Theorem 6. *For any fixed $\delta > 0$, we have* $\mathrm{OPT}(\text{PCI-LP}) = -\min_{S\subseteq V} F_{\delta,\theta}(S)$. *Furthermore, if an LP solution x has a positive objective value, then one of its threshold sets is a negative witness.*

Proof. The indicator vector of every $S \subseteq V$ is feasible for PCI-LP and has objective value $-F_{\delta,\theta}(S)$, so $\mathrm{OPT}(\text{PCI-LP}) \ge -\min_{S\subseteq V} F_{\delta,\theta}(S)$. Conversely, let x be any feasible LP solution and define $S_t = \{u \in V \mid x_u > t\}$ for $t \in [0,1]$. The layer-cake and discrete coarea identities give

$$\sum_{u\in V} \delta\omega_{\theta,u} x_u - \sum_{(u,v)\in E} |x_u - x_v| = \int_0^1 \left(\sum_{u\in S_t} \delta\omega_{\theta,u} - \mathrm{cut}(S_t) \right) dt \\ = -\int_0^1 F_{\delta,\theta}(S_t)\, dt \le -\min_{S\subseteq V} F_{\delta,\theta}(S).$$

So we have $\mathrm{OPT}(\text{PCI-LP}) \le -\min_{S\subseteq V} F_{\delta,\theta}(S)$. Together with $\mathrm{OPT}(\text{PCI-LP}) \ge -\min_{S\subseteq V} F_{\delta,\theta}(S)$, we have $\mathrm{OPT}(\text{PCI-LP}) = -\min_{S\subseteq V} F_{\delta,\theta}(S)$. If the LP objective is positive, $\int_0^1 F_{\delta,\theta}(S_t)\, dt$ is negative, so $F_{\delta,\theta}(S_t) < 0$ for some $t \in [0,1]$, and this S_t is a negative witness. □

Edge-flow dual PCI-DP**.** Following standard LP duality, for each edge $e = (u,v) \in E$, we introduce two endpoint edge-flow variables $f_{e,u}$ and $f_{e,v}$. Define the *positive residual* at every vertex $u \in V$ as $r_u(f) = \max\{\delta\omega_{\theta,u} - \sum_{e\in E: e\ni u} f_{e,u}, 0\}$. We then derive the following dual program:

$$\text{(PCI-DP)} \qquad \min_{f} \quad \sum_{u\in V} r_u(f) \\ \text{s.t.} \quad f_{e,u} + f_{e,v} = 0, \quad \forall e = (u,v) \in E, \\ |f_{e,u}| \le 1, \quad \forall e = (u,v) \in E.$$

We next provide an intuitive interpretation of PCI-DP. Following the edge-flow view of RCI-DP introduced in the previous section, each edge acts as a flow channel of capacity 1, while each vertex u has a flow requirement $\delta\omega_{\theta,u}$. Given a feasible edge flow f, the residual $r_u(f)$ measures how much of the flow requirement at u remains unsatisfied after accounting for the flow exchanged between u and its neighbors. Thus, the objective $\min_f \sum_{u\in V} r_u(f)$ of PCI-DP seeks to satisfy the flow requirements of all vertices as fully as the edge capacities permit.

If $\mathrm{OPT}(\text{PCI-DP}) > 0$, no feasible flow can satisfy all vertex requirements. By Theorem 6 and strong LP duality, this is equivalent to $\min_{S\subseteq V} F_{\delta,\theta}(S) < 0$, so the current guess δ is too large, i.e., $\delta > \phi^*_\theta$. Conversely, if $\mathrm{OPT}(\text{PCI-DP}) = 0$, there exists a feasible flow satisfying every vertex requirement and $\min_{S\subseteq V} F_{\delta,\theta}(S) = 0$, so no negative witness exists and the current estimate $\delta \le \phi^*_\theta$. Therefore, PCI-DP converts the negative-witness search into the question of whether all vertex flow requirements can be satisfied.

Quadratic residual formulation PCI-CP**.** The objective of PCI-DP is piecewise linear and nonsmooth and thus difficult to optimize efficiently using standard first-order methods. We replace it with a smooth quadratic objective that is strongly convex, yielding the following formulation:

$$\text{(PCI-CP)} \qquad \min_{f} \quad \frac{1}{2} \sum_{u\in V} r_u(f)^2 \\ \text{s.t.} \quad f_{e,u} + f_{e,v} = 0, \quad \forall e = (u,v) \in E, \\ |f_{e,u}| \le 1, \quad \forall e = (u,v) \in E.$$

We next prove that this quadratic reformulation still solves the fixed-δ subproblem and that an optimal PCI solution can be recovered from an optimal solution to PCI-CP.

Theorem 7. *For any fixed $\delta > 0$, a negative witness exists if and only if* $\mathrm{OPT}(\text{PCI-CP}) > 0$. *Moreover, let f^* be an optimal solution with* $\mathrm{OPT}(\text{PCI-CP}) > 0$. *For any $0 \le \alpha < \max_{u\in V} r_u(f^*)$, the nonempty threshold set $S_\alpha = \{u \in V \mid r_u(f^*) > \alpha\}$ satisfies $F_{\delta,\theta}(S_\alpha) < 0$, i.e., S_α is a negative witness.*

Proof. Since $r_u(f) \ge 0$, we have $\frac{1}{2}\sum_{u\in V} r_u(f)^2 = 0 \Leftrightarrow \sum_{u\in V} r_u(f) = 0$. Thus, PCI-CP and PCI-DP yield the same negative-witness decision. To recover such a witness from f^*, consider S_α. By the same edge-exchange argument as in the proof of Theorem 4, every edge crossing the boundary of S_α is saturated toward S_α. Since every vertex in S_α has a positive residual, summing over S_α and canceling the internal edge flows gives

$$\sum_{u\in S_\alpha} r_u(f^*) = \sum_{u\in S_\alpha} \delta\omega_{\theta,u} - \mathrm{cut}(S_\alpha) = -F_{\delta,\theta}(S_\alpha) > 0.$$

Therefore, S_α is a negative witness. □

5.2 Convex-Programming Algorithm PCI-FW

Let $H_\delta(f) = \frac{1}{2}\sum_{u\in V} r_u(f)^2$ denote the objective of PCI-CP. For an edge $e = (u,v) \in E$, the two feasible endpoint derivatives are

$$\frac{\partial H_\delta(f)}{\partial f_{e,u}} = r_v(f) - r_u(f), \qquad \frac{\partial H_\delta(f)}{\partial f_{e,v}} = r_u(f) - r_v(f).$$

Thus, to decrease the objective H_δ, flow should be sent toward the endpoint with the larger residual r, which intuitively helps satisfy that vertex's remaining flow requirement. In particular, if $r_u(f) = r_v(f)$, both derivatives equal zero, and the flow on e need not be changed.

We next consider the Frank–Wolfe update of PCI-FW. For each iteration t and each edge $e = (u,v) \in E$, the update direction is chosen according to the residuals at its two endpoints:

$$(\hat f^{(t)}_{e,u}, \hat f^{(t)}_{e,v}) = \begin{cases} (1,-1), & r_u^{(t-1)} > r_v^{(t-1)}, \\ (-1,1), & r_u^{(t-1)} < r_v^{(t-1)}, \\ (f^{(t-1)}_{e,u}, f^{(t-1)}_{e,v}), & r_u^{(t-1)} = r_v^{(t-1)}. \end{cases}$$

Standard Frank–Wolfe scans every edge $e \in E$ and applies $f^{(t)}_{e,u} = (1-\gamma^{(t)}) f^{(t-1)}_{e,u} + \gamma^{(t)} \hat f^{(t)}_{e,u}$ and $f^{(t)}_{e,v} = (1-\gamma^{(t)}) f^{(t-1)}_{e,v} + \gamma^{(t)} \hat f^{(t)}_{e,v}$, where $\gamma^{(t)} = 2/(t+2)$ is the step size. As $t \to \infty$, this procedure converges to an optimal solution of PCI-CP. However, scanning the entire edge set E in every iteration is clearly not strongly local.

To design a strongly-local update, we make three key observations. (1) Under the initial zero flow, only vertices in R have positive residuals: $r_u^{(0)} = \delta d_u > 0$ for $u \in R$, whereas $r_u^{(0)} = 0$ for $u \notin R$. (2) If both endpoints of an edge have zero residual, its flow need not change. (3) Intuitively, starting from zero flow, flow updates occur only near R, and thus only a small subset of vertices around R can develop positive residuals, while the vast region far from R remains zero-residual whose incident edges need not be traversed or updated. Therefore, we maintain an *active set* A containing the

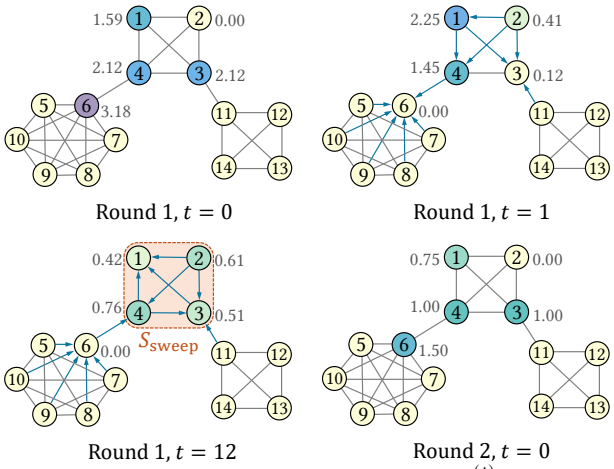
Algorithm 4: PCI-FW$(G, R, \theta, T, T_{\text{check}})$

Input: Graph $G = (V, E)$, reference set R, penalty parameter θ, maximum number of iterations T, and checking interval T_{check}.
Output: A vertex set S_{best} approximating the optimal PCI solution.

1 $S_{\text{best}} \leftarrow R$, $\delta \leftarrow \phi_\theta(S_{\text{best}}; R)$;
2 **while** *true* **do**
3 $\quad A^{(0)} \leftarrow R$, $E_A^{(0)} \leftarrow E(A^{(0)}) \cup \partial E(A^{(0)})$;
4 $\quad$ **foreach** $u \in A^{(0)}$ **do** $r_u^{(0)} \leftarrow \delta d_u$;
5 $\quad$ **foreach** $e = (u, v) \in E_A^{(0)}$ **do** $f_{e,u}^{(0)} \leftarrow 0$, $f_{e,v}^{(0)} \leftarrow 0$;
6 $\quad$ **for** $t = 1, 2, \ldots, T$ **do**
7 $\quad\quad \gamma^{(t)} \leftarrow \frac{2}{t+2}$;
8 $\quad\quad$ **foreach** $e = (u, v) \in E_A^{(t-1)}$ **do**
9 $\quad\quad\quad$ **if** $r_u^{(t-1)} > r_v^{(t-1)}$ **then** $\hat{f}_{e,u}^{(t)} \leftarrow 1$, $\hat{f}_{e,v}^{(t)} \leftarrow -1$;
10 $\quad\quad\quad$ **else if** $r_u^{(t-1)} < r_v^{(t-1)}$ **then** $\hat{f}_{e,u}^{(t)} \leftarrow -1$, $\hat{f}_{e,v}^{(t)} \leftarrow 1$;
11 $\quad\quad\quad$ **else** $\hat{f}_{e,u}^{(t)} \leftarrow f_{e,u}^{(t-1)}$, $\hat{f}_{e,v}^{(t)} \leftarrow f_{e,v}^{(t-1)}$;
12 $\quad\quad\quad f_{e,u}^{(t)} \leftarrow (1 - \gamma^{(t)}) f_{e,u}^{(t-1)} + \gamma^{(t)} \hat{f}_{e,u}^{(t)}$;
13 $\quad\quad\quad f_{e,v}^{(t)} \leftarrow (1 - \gamma^{(t)}) f_{e,v}^{(t-1)} + \gamma^{(t)} \hat{f}_{e,v}^{(t)}$;
14 $\quad\quad$ **foreach** $u \in A^{(t)} \cup \partial A^{(t)}$ **do**
15 $\quad\quad\quad r_u^{(t)} \leftarrow \max\{\delta \omega_{\theta,u} - \sum_{e \ni u} f_{e,u}^{(t)}, 0\}$
16 $\quad\quad A^{(t)} \leftarrow \{u \mid r_u^{(t)} > 0\}$, $E_A^{(t)} \leftarrow E(A^{(t)}) \cup \partial E(A^{(t)})$;
17 $\quad\quad$ **if** $t \bmod T_{\text{check}} = 0$ **then** `// Find a negative witness`
18 $\quad\quad\quad$ Sort $A^{(t)}$ as $u_1, \ldots, u_{|A^{(t)}|}$ in non-increasing order of $r_u^{(t)}$;
19 $\quad\quad\quad k^* \leftarrow \arg\min_{1 \le k \le |A^{(t)}|} \phi_\theta(\{u_1, u_2, \ldots, u_k\}; R)$;
20 $\quad\quad\quad S_{\text{sweep}} \leftarrow \{u_1, u_2, \ldots, u_{k^*}\}$;
21 $\quad\quad\quad$ **if** $\phi_\theta(S_{\text{sweep}}; R) < \delta$ **then**
22 $\quad\quad\quad\quad S_{\text{best}} \leftarrow S_{\text{sweep}}$, $\delta \leftarrow \phi_\theta(S_{\text{best}}; R)$;
23 $\quad\quad\quad\quad$ Break the "for" loop;
24 $\quad$ **if** δ *did not change in the "for" loop* **then**
25 $\quad\quad$ Break the "while" loop;
26 **return** S_{best};

vertices with positive residual. Define the *boundary edge set* of A as $\partial E(A) = \{(u, v) \in E \mid u \in A,\ v \notin A\}$. Then only the f on edges in $E(A) \cup \partial E(A)$ may need to be updated, while all other edges can be safely ignored.

Based on the above rationale, we design PCI-FW, as shown in Algorithm 4. The algorithm initializes S_{best} as R and sets the current Dinkelbach estimate to $\delta = \phi_\theta(S_{\text{best}}; R)$ (line 1). The outer "while" loop implements the Dinkelbach iteration: each round searches for a negative witness under the current δ. At the beginning of each Dinkelbach round, the algorithm initializes the active set $A^{(0)}$ and active edge set $E_A^{(0)}$ (line 3), and then initializes their flows to zero and assigns each active vertex its initial residual $r_u^{(0)} = \delta d_u$ (lines 4–5). The inner "for" loop then performs Frank–Wolfe iterations. In each iteration, it uses the step size $2/(t+2)$ (line 7), chooses an edge-wise direction by comparing the endpoint residuals (lines 9–11), and updates the edge-flow variables accordingly (lines 12–13). It then recomputes the residuals $r^{(t)}$ and refreshes the active set $A^{(t)}$ together with its active edges $E_A^{(t)}$ (lines 14–15). Every T_{check} iterations, the algorithm performs a threshold sweep over the active vertices (lines 16–19), and selects the prefix minimizing ϕ_θ as S_{sweep}. If this set has penalized conductance smaller than the current estimate, it is a negative witness, so the algorithm applies the Dinkelbach update (lines 20–22) and continues to the next outer "while" round. Otherwise, if no update occurs during the entire



The grey number beside the vertex is its residual $r_u^{(t)}$, and the blue arrows indicate flow transfers.

Figure 4: A running example of PCI-FW.

inner "for" loop, the current δ is already sufficiently small that the Frank–Wolfe search cannot find a negative witness within its iteration budget. The algorithm therefore terminates the outer loop (lines 23–24) and returns the best set S_{best} (line 25).

Example 3. *Figure 4 illustrates the execution of* PCI-FW *on the graph and reference set $R = \{1, 3, 4, 6\}$ from Figure 2c, with $\theta = 1$. Initially, S_{best} is set to R, and the first Dinkelbach round starts with $\delta = \phi_\theta(R; R) = 9/17 \approx 0.529$. At $t = 0$, all edge flows are zero, so each vertex $u \in R$ has residual $r_u^{(0)} = \delta d_u$, while every vertex outside R has zero residual. At $t = 1$, flow is transferred along the active edges toward vertices with larger residuals, as indicated by the blue arrows. These transfers reduce the residuals of the vertices in R and make the external vertex 2 active, while the residual of vertex 6 becomes zero. By $t = 12$, a sweep over the residuals returns $S_{\text{sweep}} = \{1, 2, 3, 4\}$. Its penalized conductance is $\phi_\theta(S_{\text{sweep}}; R) = 2/(11 - 3) = 1/4 < \delta$, so it is a negative witness. The algorithm therefore updates $S_{\text{best}} \leftarrow S_{\text{sweep}}$ and $\delta \leftarrow 1/4$, and then starts the second Dinkelbach round with zero flow and the new initial residuals shown in the figure. The subsequent Frank–Wolfe iterations find no negative witness with penalized conductance below $1/4$, so the algorithm terminates and returns $S_{\text{best}} = \{1, 2, 3, 4\}$, which is the optimal PCI solution.*

5.3 Strong Locality Proof

Maintaining and processing only an active vertex set A is a natural way to localize a graph algorithm. However, establishing a rigorous strong-locality guarantee for this strategy is nontrivial. To the best of our knowledge, no prior work has established strong locality for a FW method that maintains an active set, making this analysis a key theoretical contribution of our work. We next prove two key properties: (1) the volume of the active set admits a strongly-local bound, which in turn establishes that PCI-FW is strongly local; (2) processing only the active set yields a strongly local convergence guarantee independent of the global graph size.

Locality within one fixed-δ Dinkelbach round. We begin by clarifying why the two parameters θ and δ are compatible with strong locality. If the penalty parameter is too small, i.e., $\theta < \text{vol}(R)/\text{vol}(V \setminus R)$, then the entire vertex set V satisfies $\phi_\theta(V; R) = 0$ and becomes a trivial optimal PCI solution. We therefore focus on the nontrivial case $\theta \ge \text{vol}(R)/\text{vol}(V \setminus R)$. In this case, every PCI-feasible set S is a nonempty proper subset of V. Since G

is assumed to be connected and unweighted, we have $\mathrm{cut}(S) \geq 1$ and $\mathrm{vol}_\theta(S;R) \leq \mathrm{vol}(R)$. Thus, the objective value of every feasible set $\phi_\theta(S;R)$ satisfies $\phi_\theta(S;R) \geq 1/\mathrm{vol}(R)$. Furthermore, every Dinkelbach estimate δ maintained by PCI-FW is the objective value of a feasible set, and hence $\delta \geq 1/\mathrm{vol}(R)$. After excluding the trivial case, θ is a fixed input parameter and δ is lower-bounded by a quantity depending only on R. Accordingly, the strongly-local bounds below may depend on θ and δ without introducing any dependence on the global graph size $|V|$ or $|E|$.

We next show that the running time within one fixed-δ Dinkelbach round is strongly local. At iteration t, define the net vertex flow $\bar{f}_u^{(t)}$ and the total positive flow outside R as

$$\bar{f}_u^{(t)} = \sum_{e \in E: e \ni u} f_{e,u}^{(t)}, \qquad P^{(t)} = \sum_{u \notin R: \bar{f}_u^{(t)} > 0} \bar{f}_u^{(t)}.$$

The following Lemma 1 shows that the volume of the active set can be bounded in terms of $P^{(t)}$. Since the proofs are lengthy, all proofs in this subsection are provided in the appendix.

Lemma 1. *For every $t \geq 0$, the active set satisfies*

$$\mathrm{vol}(A^{(t)}) \leq (1 + \frac{1}{\delta\theta})\,\mathrm{vol}(R) + \frac{1}{\delta\theta} P^{(t)}.$$

Lemma 1 reduces the problem of controlling the entire active region to controlling $P^{(t)}$. We next derive an upper bound on $P^{(t)}$.

Lemma 2. *For every $t \geq 0$, fix $\delta > 0$ and $\theta > 0$. If $\delta\theta > 1$, then let $C_{\delta,\theta} = 0$. Otherwise, for $0 < \delta\theta \leq 1$, let $N_{\delta,\theta} = \lfloor \frac{2}{\delta\theta} \rfloor$ and*

$$C_{\delta,\theta} = \frac{6(2(1+\delta\theta)/\delta\theta)^{N_{\delta,\theta}-1}}{(N_{\delta,\theta}+2)!} - 1.$$

Then, we have $P^{(t)} \leq C_{\delta,\theta}\,\mathrm{vol}(R)$.

To give a more intuitive upper bound on $C_{\delta,\theta}$, applying the standard factorial inequality $n! \geq (n/e)^n$ gives

$$C_{\delta,\theta} \leq 6e^{\frac{2}{\delta\theta}+3} - 1.$$

Thus, $C_{\delta,\theta}$ can be regarded as a constant determined solely by δ and θ, and Lemma 2 bounds $P^{(t)}$ by a constant multiple of $\mathrm{vol}(R)$. Combining the preceding two lemmas yields Theorem 8, which provides an upper bound on the volume of the active set.

Theorem 8. *For $t \geq 0$, the active set volume can be bounded as*

$$\mathrm{vol}(A^{(t)}) \leq (1 + \frac{1 + C_{\delta,\theta}}{\delta\theta})\,\mathrm{vol}(R).$$

This result shows that, within a single fixed-δ Dinkelbach round, the active set volume $\mathrm{vol}(A^{(t)})$ admits a strongly local bound. Consequently, the local Frank–Wolfe iterations and sweep-cut checks also have strongly-local running time.

Strong locality of the complete algorithm PCI-FW**.** We next bound the total number of Dinkelbach rounds and use the lower bound $\delta \geq 1/\mathrm{vol}(R)$ to obtain a strong-locality guarantee for the complete PCI-FW algorithm.

Theorem 9. *Suppose that θ, T, and T_{check} are fixed constants. The time complexity of* PCI-FW *can be bounded in terms of* $\mathrm{vol}(R)$*. Therefore,* PCI-FW *is strongly local.*

Strongly local convergence guarantee. We next establish a strongly local convergence guarantee that is independent of the global graph sizes $|V|$ and $|E|$.

Table 1: Statistics of the real-world datasets.

Dataset	Category	$\lvert V\rvert$	$\lvert E\rvert$	Source
Retweet	Social	1.1M	2.3M	[31]
DBLP	Co-author	1.7M	8.2M	[23]
YouTube	Social	3.2M	9.4M	[23]
DBpedia	Web	3.9M	12.6M	[23]
Stack	Social	2.6M	28.2M	[23]
LiveJ	Social	4.8M	42.8M	[23]
WikiEn	Web	25.9M	543.2M	[31]
AMiner	Citation	70.2M	1.1B	[51]
Twitter	Social	52.5M	1.6B	[23]
Friend	Social	65.6M	1.8B	[48]

Theorem 10. *Let $H_\delta(f) = \frac{1}{2}\sum_{u \in V} r_u(f)^2$ denote the objective of* PCI-CP*, and let H_δ^* be its optimal value. For $t \geq 1$, we have*

$$0 \leq H_\delta(f^{(t)}) - H_\delta^* \leq \frac{16\left(1 + (1 + C_{\delta,\theta})/\delta\theta\right)^2 \mathrm{vol}(R)^2}{t+2}.$$

Therefore, the active-set strategy is not only equivalent to the global Frank–Wolfe update, but also yields a strongly local convergence guarantee.

5.4 Exact Hybrid Algorithm PCI-FW-Exact

In practice, PCI-FW achieves high approximation quality, often returning either an exact solution or one very close to optimal. Building on this advantage, we combine PCI-FW with PCI-Flow to develop an exact algorithm PCI-FW-Exact. The algorithm first runs PCI-FW to obtain S_{best} and then invokes PCI-Flow using S_{best} as a warm start. Specifically, line 1 of Algorithm 2 is replaced by $S \leftarrow S_{\mathrm{best}}$ and $\delta \leftarrow \phi_\theta(S_{\mathrm{best}};R)$, while the remaining Dinkelbach and PCI-LocalMinCut steps are executed without modification.

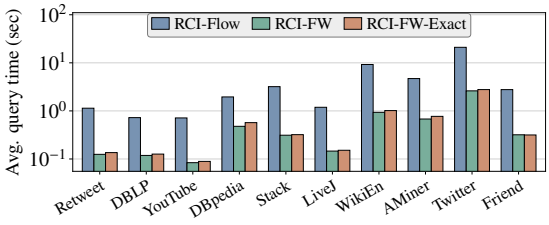
Because S_{best} is PCI-feasible, its value $\phi_\theta(S_{\mathrm{best}};R)$ is a valid upper bound on ϕ_θ^*, so this modified initialization preserves the exactness of PCI-Flow. The subsequent exact phase continues decreasing δ until the absence of a negative witness certifies optimality. In practice, PCI-FW usually places the initial estimate close to ϕ_θ^*, allowing PCI-FW-Exact to reach the certificate with substantially fewer expensive max-flow computations than PCI-Flow initialized directly from R.

6 EXPERIMENTS

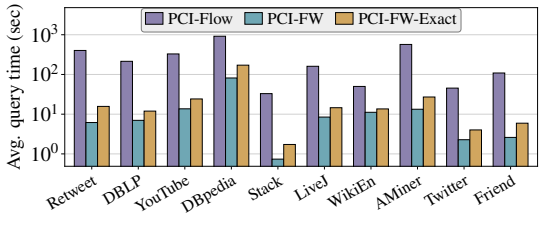
6.1 Experimental Setup

Algorithms. For the RCI problem, we compare our proposed algorithms RCI-FW (Algorithm 3) and RCI-FW-Exact (Section 4.3) against the baseline RCI-Flow (Algorithm 1). Similarly, for the PCI problem, we compare our proposed algorithms PCI-FW (Algorithm 4) and PCI-FW-Exact (Section 5.4) against the baseline PCI-Flow (Algorithm 2). The source implementations of RCI-Flow and PCI-Flow are provided by [15], while we implement all four proposed algorithms. All implementations are written in C++ and compiled with `-O3` optimization. Experiments are conducted on a Linux server equipped with two 3.0 GHz Intel Xeon Gold 6248R CPUs and 512 GB of memory. All algorithms are executed using a single thread, without multithreaded parallelization.

Parameters. For the penalty parameter θ in the PCI problem, we set $\theta = \mathrm{vol}(R)/\mathrm{vol}(V \setminus R) + 0.5$ by default. The effect of varying θ is evaluated in Exp-3. For RCI-FW, we set the maximum number of iterations to $T = 500$ and the checking interval to $T_{\mathrm{check}} = 50$. For PCI-FW, we set $T = 500$ and $T_{\mathrm{check}} = 100$. The evolution of solution quality over the Frank–Wolfe iterations is examined in Exp-5.

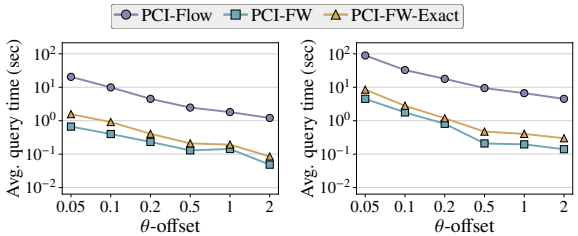


(a) Runtime comparison of the RCI algorithms.

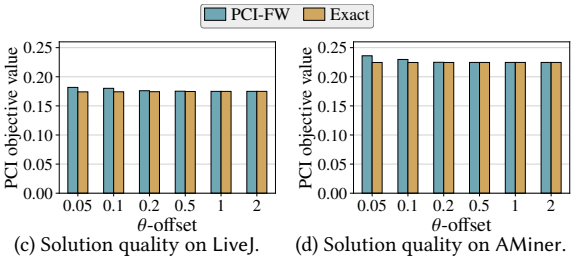


(b) Runtime comparison of the PCI algorithms.

Figure 5: Average query processing time of all algorithms on the RCI and PCI problems.



(a) Runtime on LiveJ. (b) Runtime on AMiner.



(c) Solution quality on LiveJ. (d) Solution quality on AMiner.

Figure 6: Effect of the penalty parameter θ on PCI running time and solution quality.

Datasets. We evaluate the algorithms on real-world graphs from a variety of domains, as summarized in Table 1. Twitter [23] represents follower relationships among Twitter users, whereas Retweet [31] captures retweet interactions among users. DBLP [23] is a coauthorship network among authors indexed by DBLP. YouTube [23], LiveJ [23], and Friend [48] represent friendship networks among users of YouTube, LiveJournal, and Friendster, respectively. DBpedia [23] captures RDF triple relationships among entities in DBpedia. Stack [23] represents answer and comment interactions among Stack Overflow users. WikiEn [31] is the hyperlink network among English Wikipedia pages. Finally, AMiner [51] is a paper citation network collected from AMiner. We convert each directed graph dataset into an undirected graph by ignoring edge directions and retain only the largest connected component of every graph.

Query generation. We next describe the default query-generation procedure. For each PCI query, we uniformly sample a seed vertex with degree at least 10 to avoid peripheral vertices. We use approximate personalized PageRank (APPR) followed by a sweep cut, a widely used approach for identifying a local community around a seed vertex [2]. Starting from the seed, we run APPR with stopping probability $\alpha = 0.05$ and approximation tolerance $\epsilon = 10^{-6}$ to obtain an approximate PageRank vector p. We then sort the vertices in non-increasing order of p_u/d_u and use the minimum-conductance sweep prefix P as the reference set R. We repeat this procedure to generate 64 queries for each dataset. For the RCI problem, we generate P using the same procedure and set $R = P \cup N(P)$, where $N(P)$ contains all one-hop neighbors of vertices in P. The statistics of the generated RCI and PCI queries are summarized in Table 2. Experiments with other query-generation strategies are presented in Exp-6.

6.2 Experimental Results

Exp-1: Runtime comparison in default settings. Figure 5 reports the average query processing time of all algorithms. For the RCI problem, both proposed algorithms consistently outperform the baseline RCI-Flow on all datasets. Specifically, RCI-FW achieves speedups ranging from 4.1× to 10.2×. Moreover, RCI-FW-Exact remains close in efficiency to RCI-FW, with speedups ranging from 3.4× to 10.0× over RCI-Flow. For example, on Friend, the average query processing times of RCI-Flow, RCI-FW, and RCI-FW-Exact are 2.77, 0.31, and 0.32 seconds, respectively. For the more challenging PCI problem, the advantage of the proposed algorithms is even more pronounced. PCI-FW and PCI-FW-Exact outperform the baseline on every dataset and achieve geometric-mean speedups of 24.4× and 12.7× over PCI-Flow, respectively. For example, on Retweet, algorithms PCI-Flow, PCI-FW, and PCI-FW-Exact take 402.3, 6.2, and 15.7 seconds, respectively, corresponding to speedups of 65.3× and 25.6×. Overall, these results demonstrate that the Frank–Wolfe iterations greatly accelerate both RCI and PCI query processing, while the hybrid algorithms retain most of this efficiency.

Exp-2: Effectiveness and approximation quality. Table 2 reports the objective values of the initial reference sets, the approximate solutions returned by RCI-FW and PCI-FW, and the exact optimal solutions. First, both RCI and PCI substantially improve the quality of the initial reference sets. Averaged across the ten datasets, the optimal RCI and PCI solutions reduce the objective values of R by 66.4% and 51.8%, respectively. For example, on DBpedia, the RCI conductance decreases from 0.8610 to 0.0925, while the PCI penalized conductance decreases from 0.2971 to 0.1066. These results demonstrate that both problems effectively refine the input reference sets and recover substantially lower-conductance solutions. Second, RCI-FW and PCI-FW exhibit strong approximation quality. Their objective values are nearly identical to the exact optima across all datasets: the average differences are only 0.0006 for RCI-FW and 0.0008 for PCI-FW, while the maximum differences are only 0.0013 and 0.0035, respectively. Thus, even without the subsequent exact-certification phase, the Frank–Wolfe algorithms already produce solutions very close to optimal, confirming their effectiveness as fast approximate algorithms.

Exp-3: Effect of the penalty parameter θ. Figure 6 reports the running-time and solution-quality results on LiveJ and AMiner, and similar trends are observed on other datasets. We vary the offset in $\theta = \mathrm{vol}(R)/\mathrm{vol}(V \setminus R) + \theta$-offset over $\{0.05, 0.1, 0.2, 0.5, 1, 2\}$. As shown, both proposed algorithms remain substantially faster across all tested offsets. Over the two datasets and six offset values, PCI-FW and PCI-FW-Exact are respectively 5.8–55.0× and

Table 2: Statistics and solution quality of the default queries. All entries are averaged over the 64 queries on each dataset.

S_{FW} denotes the solution returned by RCI-FW or PCI-FW, while ϕ^* and ϕ_θ^* denote the optimal RCI and PCI objective values, respectively.

Dataset	RCI queries					PCI queries				
	Avg. $\lvert R\rvert$	Avg. $\mathrm{vol}(R)$	Avg. $\phi(R)$	Avg. $\phi(S_{\mathrm{FW}})$	Avg. ϕ^*	Avg. $\lvert R\rvert$	Avg. $\mathrm{vol}(R)$	Avg. $\phi_\theta(R;R)$	Avg. $\phi_\theta(S_{\mathrm{FW}};R)$	Avg. ϕ_θ^*
Retweet	37.5K	628.1K	0.7858	0.1655	0.1642	18.4K	96.9K	0.3268	0.1406	0.1402
DBLP	21.4K	720.6K	0.7323	0.1396	0.1391	4.4K	52.5K	0.3686	0.1656	0.1647
YouTube	14.4K	1.5M	0.9462	0.1555	0.1549	10.2K	95.1K	0.4280	0.1406	0.1385
DBpedia	60.4K	2.8M	0.8610	0.0935	0.0925	32.7K	226.8K	0.2971	0.1101	0.1066
Stack	18.5K	5.3M	0.9114	0.5442	0.5437	1.2K	21.5K	0.8214	0.5603	0.5603
LiveJ	18.9K	1.7M	0.8173	0.1902	0.1899	3.7K	69.5K	0.4476	0.1752	0.1747
WikiEn	22.2K	69.6M	0.9883	0.4900	0.4899	1.7K	72.9K	0.6663	0.4470	0.4470
AMiner	67.8K	9.3M	0.8748	0.2854	0.2847	7.4K	235.3K	0.6610	0.2246	0.2246
Twitter	12.4K	200.9M	0.9973	0.5432	0.5432	1.5K	42.3K	0.8510	0.5099	0.5099
Friend	30.1K	10.1M	0.9523	0.4705	0.4699	1.6K	70.4K	0.8326	0.4803	0.4803

Table 3: Average numbers of max-flow computations and Frank–Wolfe iterations per query.

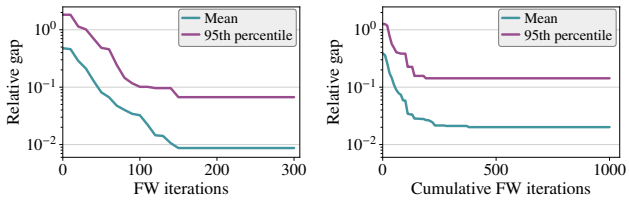
For PCI, the FW iterations are accumulated over all Dinkelbach rounds.

Dataset	RCI			PCI		
	Max-flow calls		FW iters.	Max-flow calls		FW iters.
	Flow	FW-Exact		Flow	FW-Exact	
Retweet	8.53	1.13	161.7	5.48	1.19	642.5
DBLP	8.02	1.19	126.6	5.25	1.13	712.2
YouTube	9.70	1.13	132.0	5.94	1.17	690.2
DBpedia	9.56	1.33	237.5	5.92	1.50	686.1
Stack	8.97	1.05	85.2	4.34	1.00	617.2
LiveJ	8.73	1.20	123.4	5.19	1.11	685.5
WikiEn	10.30	1.14	103.9	4.14	1.00	650.0
AMiner	10.48	1.16	124.2	7.78	1.03	668.3
Twitter	12.19	1.03	89.8	5.61	1.00	623.4
Friend	10.50	1.06	91.4	5.53	1.00	622.2

7.1–23.9× faster than PCI-Flow. Figures 6c and 6d further show that PCI-FW retains strong approximation quality as the penalty changes: across the 12 reported settings, the average and maximum differences between its objective value and the exact optimum are only 0.0027 and 0.0115, respectively. These results demonstrate that our algorithms PCI-FW and PCI-FW-Exact remain efficient and accurate across different penalty strengths.

Exp-4: Number of max-flow computations. Table 3 reports the average number of max-flow computations over the 64 queries on each dataset. Across all datasets, the two hybrid algorithms invoke max flow only slightly more than once per query: averaged across the ten datasets, RCI-FW-Exact requires 1.14 max-flow computations, compared with 9.70 for RCI-Flow, while PCI-FW-Exact requires 1.11, compared with 5.52 for PCI-Flow. This corresponds to 8.5× and 5.0× reductions, respectively. These results indicate that, for most queries, RCI-FW and PCI-FW already return an optimal solution, after which the exact phase needs only one max-flow computation to verify its optimality. Thus, the Frank–Wolfe warm start effectively eliminates most expensive Dinkelbach rounds while preserving exactness.

Exp-5: Frank–Wolfe iteration behavior. Table 3 reports the average number of FW iterations per query. For RCI-FW, the early-stopping strategy of RCI-FW reduces its average iteration count to 127.6, far below the maximum budget $T = 500$. For PCI-FW, the average iteration count across the datasets is 659.8. Typically, a small number of FW iterations in the early Dinkelbach rounds suffices to find high-quality negative witnesses and update δ, whereas the final round runs for $T = 500$ iterations without finding a negative witness and then terminates. Figure 7 further traces solution quality over all 64 queries on DBLP (similar trends are observed on other datasets). For a current solution S, we define its relative



(a) RCI-FW on DBLP. (b) PCI-FW on DBLP.

Figure 7: Mean and 95th-percentile relative gaps over the default queries during Frank–Wolfe iterations.

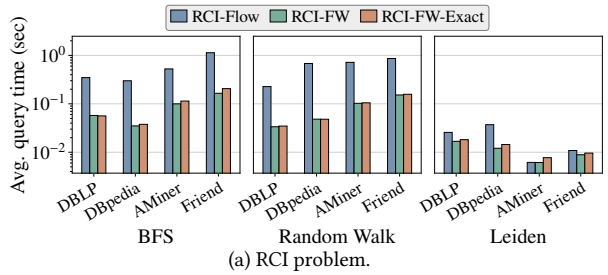
Table 4: Average objective ratios under different query-generation strategies.

Results are averaged over DBLP, DBpedia, AMiner, and Friend.

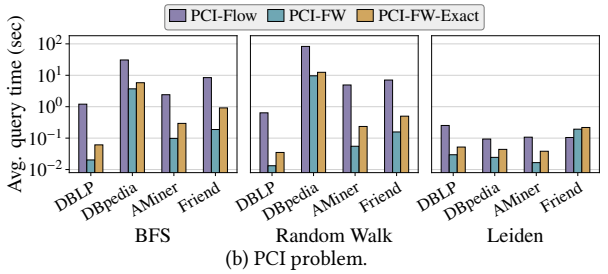
Problem	Query strategy	Avg. ratio		
		R	FW	Exact
RCI	BFS	0.8702	0.4316	0.4314
	Random Walk	0.9509	0.2244	0.2240
	Leiden	0.7318	0.0429	0.0401
PCI	BFS	0.9286	0.5869	0.5759
	Random Walk	0.9792	0.4929	0.4769
	Leiden	0.1240	0.0443	0.0401

gap as $(\phi(S) - \phi^*)/\phi^*$ for RCI and $(\phi_\theta(S;R) - \phi_\theta^*)/\phi_\theta^*$ for PCI. At iteration 0, the plotted values are the relative gaps of the initial reference sets R. We then perform a sweep-cut every ten FW iterations and report the mean and 95th-percentile relative gaps of the best sweep sets found so far. For RCI-FW, both gaps decrease rapidly during the first 100 iterations and continue to improve until stabilizing after approximately 150 iterations. For PCI-FW, most of the reduction also occurs in the first 200 iterations, after which both curves stabilize at low relative gaps. Thus, the FW iterations rapidly obtain high-quality solutions for both problems, including the more challenging queries represented by the 95th-percentile curves.

Exp-6: Evaluation under different query-generation strategies. In addition to the default APPR-based queries, we construct queries using BFS, random walks, and Leiden communities [38]. For both BFS and random walks, we first sample a seed vertex uniformly at random. BFS takes the first N vertices discovered in breadth-first order. A random walk repeatedly moves to a uniformly sampled neighbor and takes the first N vertices encountered. We set $N = 16{,}384$ for RCI and $N = 512$ for PCI, and generate 64 queries for each strategy on every dataset. For Leiden-based queries, we run the Leiden community-detection algorithm and randomly select 30 communities with 256 to 8,192 vertices. For PCI, the selected



(a) RCI problem.



(b) PCI problem.

Figure 8: Average query processing time under different query-generation strategies.

community is used directly as R. For RCI, we set R to the union of the community and all its one-hop neighbors.

Figure 8 and Table 4 report the results on four representative datasets due to space limits, and similar results are observed on the other datasets. Figure 8 shows that the proposed algorithms remain substantially faster for both BFS and random-walk queries. Across the four datasets, RCI-FW and RCI-FW-Exact achieve speedups of 5.3–14.1× and 4.6–14.2× over RCI-Flow, respectively, while PCI-FW and PCI-FW-Exact achieve speedups of 8.3–89.2× and 5.3–20.9× over PCI-Flow. The speedups are less pronounced for Leiden queries because Leiden communities generally have relatively clean boundaries, making these queries easier to solve and reducing the running times of all algorithms. Nevertheless, our methods remain competitive and achieve speedups of up to 8.5×. Table 4 further compares solution quality. For all three query strategies, both RCI and PCI substantially reduce the objective value of the initial R, and the ratios returned by the FW algorithms remain close to the exact ratios. Across the six problem–strategy combinations, the largest difference between the average FW and exact ratios is only 0.0028 for RCI and 0.0160 for PCI. These results show that our algorithms remain efficient and accurate under different query-generation strategies.

6.3 Case Studies

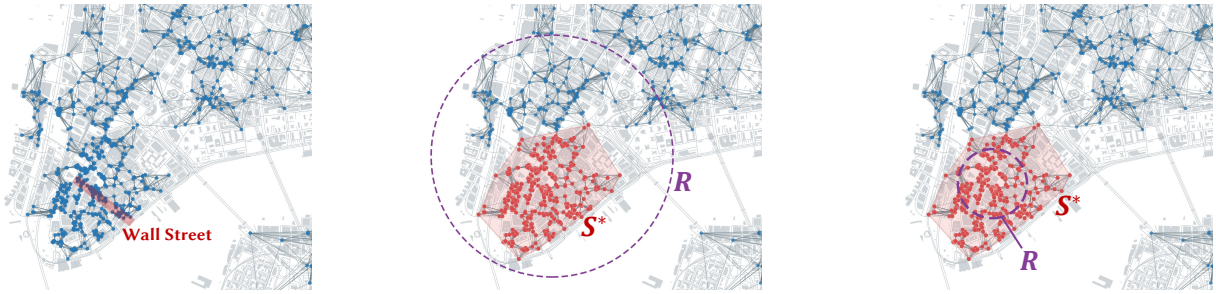
Wall Street financial-region refinement. We next study a spatial query that searches for the core financial region of Lower Manhattan around Wall Street. As shown in Figure 9a, we collect financial places including banks, insurance agencies, and investment services, and construct an undirected k-nearest-neighbor (k-NN) graph by connecting every place to its k nearest neighbors, where we set $k = 10$. For RCI, we use the places within a large circle of radius 1.00 km around Wall Street as the reference set R, allowing the algorithm to trim away loosely connected peripheral places. For PCI, we instead use a much smaller circle of radius 0.30 km as R and set $\theta = 0.5$, allowing the solution to expand beyond the initial spatial range while penalizing excessive expansion. As shown in Figures 9b and 9c, the two problems recover exactly the same densely connected region of 526 financial places. This region has only 19 edges connecting it to the rest of the graph, while containing 3,244 internal edges, resulting in an exceptionally low conductance of 0.0029. This result indicates that the business district surrounding Wall Street forms a highly cohesive financial cluster. Moreover, the approximate solutions returned by RCI-FW and PCI-FW are identical to their respective exact solutions. This case study demonstrates that RCI and PCI can refine coarse spatial range queries into coherent functional regions whose boundaries are supported by the connectivity of the underlying place network.

Amazon product-community refinement. In this case study, we examine whether conductance-based refinement can improve the recovery of a product community defined by category labels. We use the Amazon co-purchasing graph, which contains 334,863 products and 925,872 edges. For visualization, Figure 10a shows a subgraph around the ground-truth Chess category, which contains 284 products. The Chess products form a visibly cohesive group, surrounded by distinct regions associated with Poker, Film/TV, Computing, and racket sports. Starting from the chess-opening book *Meeting 1d4*, APPR returns a community that covers every Chess product but also extends into the nearby Poker region, resulting in 134 false positives among its 418 products (Figure 10b). We then refine this initial community using our RCI and PCI algorithms. For RCI, we use the union of the APPR community and all its one-hop neighbors as the reference set R, whereas for PCI, we directly use the APPR community as R and set the theta-offset to 0.5. In this case, RCI-FW, RCI-FW-Exact, PCI-FW, and PCI-FW-Exact return the same refined community, as shown in Figure 10c. The refinement removes the extension into the nearby Poker region while preserving nearly the entire Chess community. It retains 282 of the 284 ground-truth products and reduces the number of false positives from 134 to 45, increasing F1 from 0.809 to 0.923. Furthermore, the refined community has a clean boundary, with only 10 cut edges and conductance decreasing from 0.021 to 0.006. The remaining false positives reflect that co-purchasing communities do not necessarily align exactly with product categories. This case study demonstrates that our proposed algorithms can refine an initial product community by removing irrelevant products while preserving the target group, improving both structural coherence and semantic relevance.

7 RELATED WORK

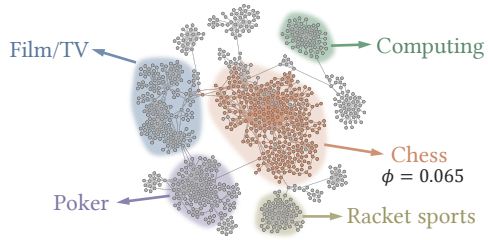
Flow-based cut improvement. Flow-based cut-improvement methods refine a reference set rather than construct a cluster from scratch. Lang and Rao introduced shrink-only refinement, corresponding to RCI [26]. Andersen and Lang allowed outputs to include vertices outside the input set [5]. Orecchia and Zhu localized this approach by bounding computation in terms of seed-related quantities [29], while Veldt, Gleich, and Mahoney simplified strong locality through adaptive flow-network expansion [41]. FlowSeed later introduced customizable penalties for excluding seed vertices and a warm-started implementation [42]. A recent survey unified and empirically compared these methods [15]. Together, this line establishes max-flow as the main tool for exact, strongly-local cut improvement. Our work instead uses strongly-local Frank–Wolfe iterations to avoid max-flow in its approximate variants and to reduce flow-based certification in its exact variants.

Local graph clustering for low-conductance cuts. Local graph clustering seeks a low-conductance set near one or more seed vertices while exploring only a small portion of the graph. Random-walk- and evolving-set-based methods explore the graph from a seed using truncated walks or a Markov chain on vertex sets, with

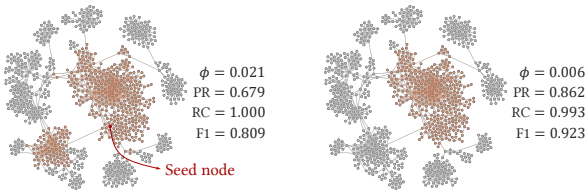


(a) Financial places and the k-NN graph ($k = 10$). (b) RCI refinement from a large region. (c) PCI refinement from a small region.

Figure 9: Spatial refinement of the core financial region around Wall Street.



(a) Ground-truth Chess community and nearby categories.



(b) Community returned by APPR. (c) Community refined by RCI/PCI. ϕ, PR, RC, and F1 denote conductance, precision, recall, and F1 score, respectively.

Figure 10: Case study on the Amazon co-purchasing graph.

running time tied to the output volume and with Cheeger-type conductance guarantees [4, 6, 35]. Diffusion-based methods compute localized vertex scores using personalized PageRank, heat kernels, or capacity-releasing diffusion, and then recover low-conductance clusters, typically through sweep cuts over those scores [2, 3, 22, 43]. Optimization- and implementation-oriented work recasts locally biased procedures through regularized optimization and parallelizes their diffusion and sweep primitives, providing complementary interpretations and scalable implementations [14, 34]. These methods construct communities from seed vertices, whereas RCI and PCI refine an arbitrary candidate set by repairing its boundary.

8 CONCLUSION

This paper develops strongly-local convex-programming algorithms for RCI and PCI. For RCI, we propose a Frank–Wolfe-based algorithm RCI-FW, which solves an exact local quadratic formulation using lightweight edge-wise Frank–Wolfe updates. For PCI, the proposed algorithm PCI-FW searches for negative witnesses through a smooth quadratic residual formulation and restricts computation to a dynamically maintained active region. We establish convergence and strong-locality guarantees for these algorithms. We further warm-start existing flow-based methods with their high-quality solutions, yielding the exact hybrid algorithms RCI-FW-Exact and PCI-FW-Exact. Experiments on ten large real-world graphs and two case studies show that the proposed algorithms substantially accelerate cluster-refinement queries while returning near-optimal or exact solutions.

REFERENCES

[1] Khaled Ammar and M. Tamer Özsu. 2018. Experimental Analysis of Distributed Graph Systems. *Proc. VLDB Endow.* 11, 10 (2018), 1151–1164. https://doi.org/10.14778/3231751.3231764

[2] Reid Andersen, Fan R. K. Chung, and Kevin J. Lang. 2006. Local Graph Partitioning using PageRank Vectors. In *47th Annual IEEE Symposium on Foundations of Computer Science, FOCS 2006, Berkeley, California, USA, 21-24 October 2006*. IEEE Computer Society, 475–486.

[3] Reid Andersen, Fan R. K. Chung, and Kevin J. Lang. 2007. Using PageRank to Locally Partition a Graph. *Internet Math.* 4, 1 (2007), 35–64.

[4] Reid Andersen, Shayan Oveis Gharan, Yuval Peres, and Luca Trevisan. 2016. Almost Optimal Local Graph Clustering Using Evolving Sets. *J. ACM* 63, 2 (2016), 15:1–15:31. https://doi.org/10.1145/2856030

[5] Reid Andersen and Kevin J. Lang. 2008. An algorithm for improving graph partitions. In *Proceedings of the Nineteenth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2008, San Francisco, California, USA, January 20-22, 2008*. SIAM, 651–660.

[6] Reid Andersen and Yuval Peres. 2009. Finding Sparse Cuts Locally Using Evolving Sets. In *Proceedings of the 41st Annual ACM Symposium on Theory of Computing, STOC 2009, Bethesda, MD, USA, May 31–June 2, 2009*. ACM, 235–244. https://doi.org/10.1145/1536414.1536449

[7] Jiazun Chen, Jun Gao, and Bin Cui. 2023. ICS-GNN$^+$: lightweight interactive community search via graph neural network. *VLDB J.* 32, 2 (2023), 447–467. https://doi.org/10.1007/S00778-022-00754-0

[8] F.R.K. Chung. 1997. *Spectral Graph Theory*. American Mathematical Society.

[9] Yizhou Dai, Miao Qiao, and Lijun Chang. 2022. Anchored Densest Subgraph. In *SIGMOD '22: International Conference on Management of Data, Philadelphia, PA, USA, June 12 - 17, 2022*, Zachary G. Ives, Angela Bonifati, and Amr El Abbadi (Eds.). ACM, 1200–1213. https://doi.org/10.1145/3514221.3517890

[10] Bhavana Bharat Dalvi, Meghana Kshirsagar, and S. Sudarshan. 2008. Keyword search on external memory data graphs. *Proc. VLDB Endow.* 1, 1 (2008), 1189–1204. https://doi.org/10.14778/1453856.1453982

[11] Srayan Datta and Eytan Adar. 2018. CommunityDiff: Visualizing Community Clustering Algorithms. *ACM Trans. Knowl. Discov. Data* 12, 1 (2018), 11:1–11:34. https://doi.org/10.1145/3047009

[12] Yefim Dinitz. 1970. Algorithm for Solution of a Problem of Maximum Flow in Networks with Power Estimation. *Soviet Math. Dokl.* 11 (01 1970), 1277–1280.

[13] Charles M. Fiduccia and Robert M. Mattheyses. 1982. A linear-time heuristic for improving network partitions. In *Proceedings of the 19th Design Automation Conference, DAC '82, Las Vegas, Nevada, USA, June 14-16, 1982*, James S. Crabbe, Charles E. Radke, and Hillel Ofek (Eds.). ACM/IEEE, 175–181. https://doi.org/10.1145/800263.809204

[14] Kimon Fountoulakis, David F. Gleich, and Michael W. Mahoney. 2017. An Optimization Approach to Locally-Biased Graph Algorithms. *Proc. IEEE* 105, 2 (2017), 256–272.

[15] Kimon Fountoulakis, Meng Liu, David F. Gleich, and Michael W. Mahoney. 2023. Flow-Based Algorithms for Improving Clusters: A Unifying Framework, Software, and Performance. *SIAM Rev.* 65, 1 (2023), 59–143. https://doi.org/10.1137/20M1333055

[16] Nadime Francis, Alastair Green, Paolo Guagliardo, Leonid Libkin, Tobias Lindaaker, Victor Marsault, Stefan Plantikow, Mats Rydberg, Petra Selmer, and Andrés Taylor. 2018. Cypher: An Evolving Query Language for Property Graphs. In *Proceedings of the 2018 International Conference on Management of Data, SIGMOD Conference 2018, Houston, TX, USA, June 10-15, 2018*, Gautam Das, Christopher M. Jermaine, and Philip A. Bernstein (Eds.). ACM, 1433–1445. https://doi.org/10.1145/3183713.3190657

[17] Jun Gao, Jiazun Chen, Zhao Li, and Ji Zhang. 2021. ICS-GNN: Lightweight Interactive Community Search via Graph Neural Network. *Proc. VLDB Endow.* 14, 6 (2021), 1006–1018. https://doi.org/10.14778/3447689.3447704

[18] Alexandra Henzinger, Alexander Noe, and Christian Schulz. 2020. ILP-Based Local Search for Graph Partitioning. *ACM J. Exp. Algorithmics* 25 (2020), 1–26. https://doi.org/10.1145/3398634

[19] Yufan Huang, David F. Gleich, and Nate Veldt. 2024. Densest Subhypergraph: Negative Supermodular Functions and Strongly Localized Methods. In *Proceedings of the ACM on Web Conference 2024, WWW 2024, Singapore, May 13-17, 2024*, Tat-Seng Chua, Chong-Wah Ngo, Ravi Kumar, Hady W. Lauw, and Roy Ka-Wei Lee (Eds.). ACM, 881–892. https://doi.org/10.1145/3589334.3645624

[20] Martin Jaggi. 2013. Revisiting Frank-Wolfe: Projection-Free Sparse Convex Optimization. In *Proceedings of the 30th International Conference on Machine Learning, ICML 2013, Atlanta, GA, USA, 16-21 June 2013 (JMLR Workshop and Conference Proceedings, Vol. 28)*. JMLR.org, 427–435. http://proceedings.mlr.press/v28/jaggi13.html

[21] Minhao Jiang, Ada Wai-Chee Fu, and Raymond Chi-Wing Wong. 2015. Exact Top-k Nearest Keyword Search in Large Networks. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, Melbourne, Victoria, Australia, May 31 - June 4, 2015*, Timos K. Sellis, Susan B. Davidson, and Zachary G. Ives (Eds.). ACM, 393–404. https://doi.org/10.1145/2723372.2749447

[22] Kyle Kloster and David F. Gleich. 2014. Heat Kernel Based Community Detection. In *The 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD 2014, New York, NY, USA, August 24–27, 2014*. ACM, 1386–1395. https://doi.org/10.1145/2623330.2623706

[23] Jérôme Kunegis. 2013. KONECT – The Koblenz Network Collection. In *Proc. Int. Conf. on World Wide Web Companion*. 1343–1350. http://dl.acm.org/citation.cfm?id=2488173

[24] Ming-Jun Lai and Daniel McKenzie. 2020. Compressive Sensing for Cut Improvement and Local Clustering. *SIAM J. Math. Data Sci.* 2, 2 (2020), 368–395. https://doi.org/10.1137/19M1265971

[25] Andrea Lancichinetti, Santo Fortunato, and János Kertész. 2009. Detecting the overlapping and hierarchical community structure in complex networks. *New Journal of Physics* 11, 3 (March 2009), 033015. https://doi.org/10.1088/1367-2630/11/3/033015

[26] Kevin J. Lang and Satish Rao. 2004. A Flow-Based Method for Improving the Expansion or Conductance of Graph Cuts. In *Integer Programming and Combinatorial Optimization, 10th International IPCO Conference, New York, NY, USA, June 7-11, 2004, Proceedings (Lecture Notes in Computer Science, Vol. 3064)*. Springer, 325–337.

[27] Yushi Liu, Liwei Yuan, Zhihao Chen, Yekai Yu, Zhao Zhang, Cheqing Jin, and Ying Yan. 2023. ChainDash: An Ad-Hoc Blockchain Data Analytics System. *Proc. VLDB Endow.* 16, 12 (2023), 4022–4025. https://doi.org/10.14778/3611540.3611611

[28] Zongqing Ma, Xi Wu, Qi Song, Yong Luo, Yan Wang, and Jiliu Zhou. 2018. Automated nasopharyngeal carcinoma segmentation in magnetic resonance images by combination of convolutional neural networks and graph cut. *Experimental and therapeutic medicine* 16, 3 (2018), 2511–2521.

[29] Lorenzo Orecchia and Zeyuan Allen Zhu. 2014. Flow-Based Algorithms for Local Graph Clustering. In *Proceedings of the Twenty-Fifth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2014, Portland, Oregon, USA, January 5-7, 2014*. SIAM, 1267–1286. https://doi.org/10.1137/1.9781611973402.94

[30] Yanjun Qi, Fernanda Balem, Christos Faloutsos, Judith Klein-Seetharaman, and Ziv Bar-Joseph. 2008. Protein Complex Identification by Supervised Graph Local Clustering. In *Proceedings of the 16th International Conference on Intelligent Systems for Molecular Biology, ISMB 2008, Toronto, Canada, July 19–23, 2008*. 250–268. https://doi.org/10.1093/BIOINFORMATICS/BTN164

[31] Ryan A. Rossi and Nesreen K. Ahmed. 2015. The Network Data Repository with Interactive Graph Analytics and Visualization. In *AAAI*. https://networkrepository.com

[32] Peter Sanders and Christian Schulz. 2011. Engineering Multilevel Graph Partitioning Algorithms. In *Algorithms - ESA 2011 - 19th Annual European Symposium, Saarbrücken, Germany, September 5-9, 2011. Proceedings (Lecture Notes in Computer Science, Vol. 6942)*, Camil Demetrescu and Magnús M. Halldórsson (Eds.). Springer, 469–480. https://doi.org/10.1007/978-3-642-23719-5_40

[33] Satu Elisa Schaeffer. 2007. Graph Clustering. *Comput. Sci. Rev.* 1, 1 (2007), 27–64. https://doi.org/10.1016/J.COSREV.2007.05.001

[34] Julian Shun, Farbod Roosta-Khorasani, Kimon Fountoulakis, and Michael W. Mahoney. 2016. Parallel Local Graph Clustering. *Proc. VLDB Endow.* 9, 12 (2016), 1041–1052.

[35] Daniel A. Spielman and Shang-Hua Teng. 2013. A Local Clustering Algorithm for Massive Graphs and Its Application to Nearly Linear Time Graph Partitioning. *SIAM J. Comput.* 42, 1 (2013), 1–26. https://doi.org/10.1137/080744888

[36] Longxu Sun, Xin Huang, Jiannan Wang, and Jianliang Xu. 2025. A Flexible Framework for Query-oriented Interactive Community Search. *Proc. VLDB Endow.* 18, 6 (2025), 1977–1990. https://doi.org/10.14778/3725688.3725720

[37] Yuhan Sun and Mohamed Sarwat. 2021. Riso-Tree: An Efficient and Scalable Index for Spatial Entities in Graph Database Management Systems. *ACM Trans. Spatial Algorithms Syst.* 7, 3 (2021), 12:1–12:39. https://doi.org/10.1145/3450945

[38] Vincent A. Traag, Ludo Waltman, and Nees Jan van Eck. 2018. From Louvain to Leiden: guaranteeing well-connected communities. *CoRR* abs/1810.08473 (2018). arXiv:1810.08473 http://arxiv.org/abs/1810.08473

[39] Matee Ullah, Adnan Iltaf, Qiujun Hou, Farman Ali, and Chuancai Liu. 2018. A Foreground Extraction Approach Using Convolutional Neural Network with Graph Cut. In *2018 IEEE 3rd International Conference on Image, Vision and Computing (ICIVC)*. 40–44. https://doi.org/10.1109/ICIVC.2018.8492887

[40] Oron Vanunu, Oded Magger, Eytan Ruppin, Tomer Shlomi, and Roded Sharan. 2010. Associating Genes and Protein Complexes with Disease via Network Propagation. *PLoS Comput. Biol.* 6, 1 (2010). https://doi.org/10.1371/JOURNAL.PCBI.1000641

[41] Nate Veldt, David F. Gleich, and Michael W. Mahoney. 2016. A Simple and Strongly-Local Flow-Based Method for Cut Improvement. In *Proceedings of the 33rd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19-24, 2016 (JMLR Workshop and Conference Proceedings, Vol. 48)*. JMLR.org, 1938–1947.

[42] Nate Veldt, Christine Klymko, and David F. Gleich. 2019. Flow-Based Local Graph Clustering with Better Seed Set Inclusion. In *Proceedings of the 2019 SIAM International Conference on Data Mining, SDM 2019, Calgary, Alberta, Canada, May 2–4, 2019*. SIAM, 378–386. https://doi.org/10.1137/1.9781611975673.43

[43] Di Wang, Kimon Fountoulakis, Monika Henzinger, Michael W. Mahoney, and Satish Rao. 2017. Capacity Releasing Diffusion for Speed and Locality. In *Proceedings of the 34th International Conference on Machine Learning, ICML 2017, Sydney, NSW, Australia, 6–11 August 2017 (Proceedings of Machine Learning Research, Vol. 70)*. PMLR, 3598–3607.

[44] Joyce Jiyoung Whang, David F. Gleich, and Inderjit S. Dhillon. 2016. Overlapping Community Detection Using Neighborhood-Inflated Seed Expansion. *IEEE Trans. Knowl. Data Eng.* 28, 5 (2016), 1272–1284. https://doi.org/10.1109/TKDE.2016.2518687

[45] Yubao Wu, Ruoming Jin, Jing Li, and Xiang Zhang. 2015. Robust Local Community Detection: On Free Rider Effect and Its Elimination. *Proc. VLDB Endow.* 8, 7 (2015), 798–809. https://doi.org/10.14778/2752939.2752948

[46] Yubao Wu, Ruoming Jin, and Xiang Zhang. 2014. Fast and unified local search for random walk based k-nearest-neighbor query in large graphs. In *International Conference on Management of Data, SIGMOD 2014, Snowbird, UT, USA, June 22-27, 2014*, Curtis E. Dyreson, Feifei Li, and M. Tamer Özsu (Eds.). ACM, 1139–1150. https://doi.org/10.1145/2588555.2610500

[47] Jierui Xie, Stephen Kelley, and Boleslaw K. Szymanski. 2013. Overlapping community detection in networks: The state-of-the-art and comparative study. *ACM Comput. Surv.* 45, 4 (2013), 43:1–43:35. https://doi.org/10.1145/2501654.2501657

[48] Jaewon Yang and Jure Leskovec. 2015. Defining and evaluating network communities based on ground-truth. *Knowl. Inf. Syst.* 42, 1 (2015), 181–213. https://doi.org/10.1007/S10115-013-0693-Z

[49] Xiaowei Ye, Rong-Hua Li, Lei Liang, Zhizhen Liu, Longlong Lin, and Guoren Wang. 2024. Efficient and Effective Anchored Densest Subgraph Search: A Convex-programming based Approach. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2024, Barcelona, Spain, August 25-29, 2024*, Ricardo Baeza-Yates and Francesco Bonchi (Eds.). ACM, 3907–3918. https://doi.org/10.1145/3637528.3671727

[50] Dalong Zhang, Xin Huang, Ziqi Liu, Jun Zhou, Zhiyang Hu, Xianzheng Song, Zhibang Ge, Lin Wang, Zhiqiang Zhang, and Yuan Qi. 2020. AGL: A Scalable System for Industrial-purpose Graph Machine Learning. *Proc. VLDB Endow.* 13, 12 (2020), 3125–3137. https://doi.org/10.14778/3415478.3415539

[51] Fanjin Zhang, Xiao Liu, Jie Tang, Yuxiao Dong, Peiran Yao, Jie Zhang, Xiaotao Gu, Yan Wang, Evgeny Kharlamov, Bin Shao, Rui Li, and Kuansan Wang. 2023. OAG: Linking Entities Across Large-Scale Heterogeneous Knowledge Graphs. *IEEE Trans. Knowl. Data Eng.* 35, 9 (2023), 9225–9239. https://doi.org/10.1109/TKDE.2022.3222168

[52] Qi Zhang, Yalong Zhang, Ronghua Li, and Guoren Wang. 2024. Approximate Anchored Densest Subgraph Search on Large Static and Dynamic Graphs. *Proc. VLDB Endow.* 18, 3 (2024), 623–636. https://doi.org/10.14778/3712221.3712230

[53] Jiří Šíma and Satu Schaeffer. 2005. On the NP-Completeness of Some Graph Cluster Measures. 530–537. https://doi.org/10.1007/11611257_51

APPENDIX

A PROOF OF THEOREM 5

Proof. Let B be the signed incidence matrix such that, for each edge $e = (u, v) \in E_R$, $B_{u,e} = 1$, $B_{v,e} = -1$, and $B_{w,e} = 0$ for every $w \notin \{u, v\}$. Define $q_e = f_{e,u} = -f_{e,v}$ for each $e = (u, v) \in E_R$. With $D = \mathrm{diag}(d_u : u \in R)$ and $\mathcal{F} = [-1, 1]^{|E_R|}$, we have $\rho(q) = D^{-1}(b + Bq)$ and $Q(q) = \frac{1}{2}(b + Bq)^\top D^{-1}(b + Bq)$. Let q^* minimize Q over $\mathcal{F}$, and write $\rho^* = \rho(q^*)$.

We first bound the Frank–Wolfe curvature constant C_Q. For an L-smooth function over a compact domain, the standard smoothness–diameter relation gives $C_Q \le L\,\mathrm{diam}(\mathcal{F})^2$ [20]. Here, $\nabla^2 Q(q) = B^\top D^{-1} B$. Let d_u^R be the degree of u in the induced subgraph of R. For any vector $z \in \mathbb{R}^{|E_R|}$, the standard normalized-Laplacian estimate, obtained from Cauchy–Schwarz and $d_u^R \le d_u$, gives [8]

$$z^\top B^\top D^{-1} B z = \sum_{u \in R} \frac{((Bz)_u)^2}{d_u} \le \sum_{u \in R} \frac{d_u^R}{d_u} \sum_{e \in E_R : e \ni u} z_e^2 \le 2\|z\|_2^2.$$

Thus, $\|B^\top D^{-1}B\|_2 \le 2$, and Q is 2-smooth under the Euclidean norm. Moreover, $\mathcal{F} = [-1,1]^{|E_R|}$ has squared diameter $4|E_R|$. Therefore,

$$C_Q \le 2 \cdot 4|E_R| = 8|E_R| \le 4\,\mathrm{vol}(R).$$

Define $h_t = Q(q^{(t)}) - Q(q^*)$. Since $q^{(0)} = 0$, $b_u \le d_u$, and $\sum_{u\in R} b_u = \mathrm{cut}(R)$, we have $h_0 \le Q(0) \le \mathrm{vol}(R)/2$. The standard curvature inequality, the linear-oracle property, and the convexity of Q imply $h_t \le (1-\gamma^{(t)})h_{t-1} + (\gamma^{(t)})^2 C_Q/2$ [20]. Substituting $\gamma^{(t)} = 2/(t+2)$ and unrolling this recurrence yields

$$h_t \le \frac{2h_0 + 2C_Q\sum_{i=1}^{t}\frac{i+1}{i+2}}{(t+1)(t+2)} \le \frac{2h_0 + 2tC_Q}{(t+1)(t+2)} \le \frac{8\,\mathrm{vol}(R)}{t}.$$

We next convert this objective gap into a minimum-load gap. The set of attainable load vectors is convex, and ρ^* minimizes $\frac{1}{2}\|\rho\|_D^2$ over this set, where $\|z\|_D^2 = \sum_{u\in R} d_u z_u^2$. Its first-order optimality condition therefore gives, for every feasible q, $Q(q) - Q(q^*) \ge \frac{1}{2}\|\rho(q) - \rho^*\|_D^2$. Let $\widehat{u} \in \arg\min_{u\in R}\rho_u(q^{(t)})$. The pair $(\rho^{(t)}_{\min}, q^{(t)})$ is feasible for RCI-DP, so $\rho^{(t)}_{\min} \le \phi^*$. Moreover, the exactness of RCI-CP gives $\min_u \rho^*_u = \phi^*$ and hence $\rho^*_{\widehat{u}} \ge \phi^*$. By the definition of the weighted norm,

$$d_{\widehat{u}}\big(\rho^*_{\widehat{u}} - \rho_{\widehat{u}}(q^{(t)})\big)^2 \le \sum_{u\in R} d_u\big(\rho^*_u - \rho_u(q^{(t)})\big)^2 = \|\rho(q^{(t)}) - \rho^*\|_D^2.$$

Since $d_{\widehat{u}} \ge d_{\min}$, this implies

$$\rho^*_{\widehat{u}} - \rho_{\widehat{u}}(q^{(t)}) \le \frac{\|\rho(q^{(t)}) - \rho^*\|_D}{\sqrt{d_{\widehat{u}}}} \le \frac{\|\rho(q^{(t)}) - \rho^*\|_D}{\sqrt{d_{\min}}}.$$

Applying the preceding objective-gap inequality to $q^{(t)}$ and using $h_t \le 8\,\mathrm{vol}(R)/t$, we further obtain

$$\frac{\|\rho(q^{(t)}) - \rho^*\|_D}{\sqrt{d_{\min}}} \le \sqrt{\frac{2h_t}{d_{\min}}} \le 4\sqrt{\frac{\mathrm{vol}(R)}{d_{\min}t}}.$$

Combining these inequalities proves the first claim of the theorem.

It remains to analyze the sweep cut. For any feasible q, let $s(q)$ be its Frank–Wolfe linear-oracle solution, i.e., $s_e(q) = 1$ if $\rho_u(q) < \rho_v(q)$ and $s_e(q) = -1$ otherwise for each edge $e = (u,v) \in E_R$. The corresponding Frank–Wolfe gap is $g(q) = \sum_{e=(u,v)\in E_R}\big(\rho_u(q) - \rho_v(q)\big)\big(q_e - s_e(q)\big)$. Also define $\rho_{\min}(q) = \min_u \rho_u(q)$, $M_{\min}(q) = \{u \mid \rho_u(q) = \rho_{\min}(q)\}$, $v_{\min}(q) = \mathrm{vol}(M_{\min}(q))$, and

$$S_\alpha(q) = \{u \in R \mid \rho_u(q) \le \alpha\}, \qquad \widehat{\phi}(q) = \min_{\alpha: S_\alpha(q)\neq\emptyset}\phi(S_\alpha(q)).$$

We first show the sweep-rounding inequality

$$\widehat{\phi}(q) - \rho_{\min}(q) \le \sqrt{\frac{g(q)}{v_{\min}(q)}} \le \sqrt{\frac{g(q)}{d_{\min}}}.$$

The claim is immediate if $\widehat{\phi}(q) = \rho_{\min}(q)$. Otherwise, take $0 < \epsilon < \widehat{\phi}(q) - \rho_{\min}(q)$, set $\alpha = \rho_{\min}(q) + \epsilon$, and let $x_u = \max\{\alpha - \rho_u(q), 0\}$. For any $\lambda > 0$ such that $X_\lambda = \{u \in R \mid x_u \ge \lambda\}$ is nonempty, the definition of x gives $X_\lambda = \{u \in R \mid \rho_u(q) \le \alpha - \lambda\}$. Thus, X_λ is a ρ-threshold set and satisfies $\phi(X_\lambda) \ge \widehat{\phi}(q)$. Moreover, since $\epsilon < \widehat{\phi}(q) - \rho_{\min}(q)$, we have $\alpha = \rho_{\min}(q) + \epsilon < \widehat{\phi}(q)$, and hence $\phi(X_\lambda) > \alpha$. Thus, the layer-cake and coarea identities imply $C(x) \ge \alpha\mathcal{V}(x)$, where $C(x) = \sum_{u\in R} b_u x_u + \sum_{(u,v)\in E_R}|x_u - x_v|$ and $\mathcal{V}(x) = \sum_{u\in R} d_u x_u$.

We next relate $C(x)$ to the Frank–Wolfe gap. From the definition of $\rho_u(q)$, we have $d_u\rho_u(q) = b_u + \sum_{e\in E_R: e\ni u} f_{e,u}$. Using $f_{e,u} + f_{e,v} = 0$ on every edge, we obtain

$$\begin{aligned}\sum_{u\in R} d_u\rho_u(q)x_u &= \sum_{u\in R} b_u x_u + \sum_{e=(u,v)\in E_R}\big(f_{e,u}x_u + f_{e,v}x_v\big)\\ &= \sum_{u\in R} b_u x_u + \sum_{e=(u,v)\in E_R} f_{e,u}(x_u - x_v).\end{aligned}$$

Subtracting this identity from the definition of $C(x)$ cancels the boundary terms and gives

$$C(x) - \sum_{u\in R} d_u\rho_u(q)x_u = \sum_{e=(u,v)\in E_R}\big(|x_u - x_v| - f_{e,u}(x_u - x_v)\big).$$

For each edge, label its endpoints so that $\rho_u(q) \le \rho_v(q)$. Since $x_u = \max\{\alpha - \rho_u(q), 0\}$ is non-increasing and 1-Lipschitz in $\rho_u(q)$, we have $0 \le x_u - x_v \le \rho_v(q) - \rho_u(q)$. Hence, the contribution of this edge to the preceding sum satisfies

$$|x_u - x_v| - f_{e,u}(x_u - x_v) = (1 - f_{e,u})(x_u - x_v) \le (1 - f_{e,u})(\rho_v(q) - \rho_u(q)).$$

When $\rho_u(q) \le \rho_v(q)$, the linear oracle sends the edge flow toward u, and the corresponding edge contribution to $g(q)$ is $(\rho_u(q) - \rho_v(q))(f_{e,u} - 1) = (1 - f_{e,u})(\rho_v(q) - \rho_u(q))$. Summing the above edge-wise inequality therefore yields

$$C(x) - \sum_{u\in R} d_u\rho_u(q)x_u \le g(q).$$

Since $\rho_u(q) = \alpha - x_u$ whenever $x_u > 0$, the left-hand side together with $C(x) \ge \alpha\mathcal{V}(x)$ gives $\|x\|_D^2 \le g(q)$. Every vertex in $M_{\min}(q)$ satisfies $x_u = \epsilon$, so $v_{\min}(q)\epsilon^2 \le g(q)$. Since this inequality holds for every $0 < \epsilon < \widehat{\phi}(q) - \rho_{\min}(q)$, taking the limit as ϵ approaches $\widehat{\phi}(q) - \rho_{\min}(q)$ from below gives $v_{\min}(q)(\widehat{\phi}(q) - \rho_{\min}(q))^2 \le g(q)$. Therefore, $\widehat{\phi}(q) - \rho_{\min}(q) \le \sqrt{g(q)/v_{\min}(q)}$, proving the sweep-rounding inequality.

Finally, let $g_t = g(q^{(t)})$. For every $\theta \in [0,1]$, the curvature inequality and the optimality of q^* imply $h_t \ge \theta g_t - \theta^2 C_Q/2$. For $t \ge 4$, taking $\theta = 2/\sqrt{t}$ and using $h_t \le 8\,\mathrm{vol}(R)/t$ and $C_Q \le 4\,\mathrm{vol}(R)$ gives $g_t \le 8\,\mathrm{vol}(R)/\sqrt{t}$. The prefix sweep in Algorithm 3 is at least as good as the best threshold set, and hence

$$0 \le \phi(S^{(t)}_{\mathrm{sweep}}) - \phi^* \le \widehat{\phi}(q^{(t)}) - \rho^{(t)}_{\min} \le \sqrt{\frac{g_t}{d_{\min}}} \le 2\sqrt{\frac{2\,\mathrm{vol}(R)}{d_{\min}}}\,t^{-1/4}.$$

For $1 \le t < 4$, the same bound follows from $0 \le \phi(S^{(t)}_{\mathrm{sweep}}) - \phi^* \le 1$ and $\mathrm{vol}(R)/d_{\min} \ge 1$. This proves the second claim. □

B PROOFS FOR SECTION 5.3

B.1 Proof of Lemma 1

Proof. Let $A^{(t)}_{\mathrm{in}} = A^{(t)} \cap R$ and $A^{(t)}_{\mathrm{out}} = A^{(t)} \setminus R$. Clearly, $\mathrm{vol}(A^{(t)}_{\mathrm{in}}) \le \mathrm{vol}(R)$. For $u \notin R$, the definition of the residual gives $r^{(t)}_u = \max\{-\delta\theta d_u - \tilde{f}^{(t)}_u, 0\}$. Hence, if $u \in A^{(t)}_{\mathrm{out}}$, then $\tilde{f}^{(t)}_u < -\delta\theta d_u$. Define the total negative net flow outside R as $N^{(t)} = \sum_{u\notin R}\max\{-\tilde{f}^{(t)}_u, 0\}$. Summing the preceding active-vertex condition over $A^{(t)}_{\mathrm{out}}$ yields

$$\delta\theta\,\mathrm{vol}(A^{(t)}_{\mathrm{out}}) \le N^{(t)}. \tag{1}$$

We next bound $N^{(t)}$ using flow conservation. Define the positive and negative net flows inside R as

$$P^{(t)}_R = \sum_{u\in R}\max\{\tilde{f}^{(t)}_u, 0\}, \qquad N^{(t)}_R = \sum_{u\in R}\max\{-\tilde{f}^{(t)}_u, 0\}.$$

The antisymmetry of every edge flow implies

$$\sum_{u\in V}\tilde{f}^{(t)}_u = \sum_{e=(u,v)\in E}(f^{(t)}_{e,u} + f^{(t)}_{e,v}) = 0.$$

Therefore, the total positive and negative net flows are equal, i.e., $P_R^{(t)} + P^{(t)} = N_R^{(t)} + N^{(t)}$, and hence

$$N^{(t)} = P_R^{(t)} + P^{(t)} - N_R^{(t)} \le P_R^{(t)} + P^{(t)}.$$

Since $f_{e,u}^{(t)} \le 1$ for every incident edge, $\bar{f}_u^{(t)} \le d_u$ and thus $P_R^{(t)} \le \sum_{u \in R} d_u = \mathrm{vol}(R)$. Substituting this bound into Equation (1) gives

$$\mathrm{vol}(A_{\mathrm{out}}^{(t)}) \le \frac{\mathrm{vol}(R) + P^{(t)}}{\delta\theta}.$$

Combining this inequality with $\mathrm{vol}(A_{\mathrm{in}}^{(t)}) \le \mathrm{vol}(R)$ proves

$$\mathrm{vol}(A^{(t)}) \le \left(1 + \frac{1}{\delta\theta}\right)\mathrm{vol}(R) + \frac{P^{(t)}}{\delta\theta}.$$

□

B.2 Proof of Lemma 2

Proof. We first examine how positive net flow can arise outside R. Consider a vertex $u \notin R$ that is inactive at iteration $t-1$. If an incident edge $e = (u, v)$ is outside $E_A^{(t-1)}$, its flow remains unchanged. Otherwise, v is active and $r_u^{(t-1)} = 0 < r_v^{(t-1)}$, so the Frank–Wolfe direction at the u endpoint is $\hat{f}_{e,u}^{(t)} = -1$. Since $f_{e,u}^{(t-1)} \ge -1$,

$$f_{e,u}^{(t)} = (1 - \gamma^{(t)}) f_{e,u}^{(t-1)} - \gamma^{(t)} \le f_{e,u}^{(t-1)}.$$

Thus, $\bar{f}_u^{(t)} \le \bar{f}_u^{(t-1)}$, and an inactive external vertex cannot increase its contribution to $P^{(t)}$.

Now consider $u \in A_{\mathrm{out}}^{(t-1)}$. All its incident edges belong to $E_A^{(t-1)}$, and the active condition gives $\bar{f}_u^{(t-1)} < -\delta\theta d_u$. Since $\hat{f}_{e,u}^{(t)} \le 1$ for every incident edge,

$$\begin{aligned}\bar{f}_u^{(t)} &= (1 - \gamma^{(t)})\bar{f}_u^{(t-1)} + \gamma^{(t)} \sum_{e \ni u} \hat{f}_{e,u}^{(t)} \\ &\le -(1 - \gamma^{(t)})\delta\theta d_u + \gamma^{(t)} d_u.\end{aligned}$$

With $\gamma^{(t)} = 2/(t+2)$, define

$$a_t = \max\left\{0, \frac{2 - \delta\theta t}{\delta\theta(t+2)}\right\}.$$

The preceding inequality implies $\max\{\bar{f}_u^{(t)}, 0\} \le a_t \delta\theta d_u$. The external vertices inactive at iteration $t-1$ cannot increase their positive net flow, so their total contribution to $P^{(t)}$ is at most their previous contribution in $P^{(t-1)}$. Hence, any additional positive net flow can only come from vertices in $A_{\mathrm{out}}^{(t-1)}$. As shown above, each such vertex u contributes at most $a_t \delta\theta d_u$. Summing over these vertices gives

$$\begin{aligned}P^{(t)} &\le P^{(t-1)} + a_t \delta\theta \,\mathrm{vol}(A_{\mathrm{out}}^{(t-1)}) \\ &\le P^{(t-1)} + a_t\big(\mathrm{vol}(R) + P^{(t-1)}\big),\end{aligned}$$

where the last inequality follows from Equation (1) and $N^{(t-1)} \le \mathrm{vol}(R) + P^{(t-1)}$. Equivalently,

$$P^{(t)} + \mathrm{vol}(R) \le (1 + a_t)\big(P^{(t-1)} + \mathrm{vol}(R)\big). \tag{2}$$

The initialization $f^{(0)} = 0$ gives $P^{(0)} = 0$. Moreover, no external vertex is active initially, so the first part of the proof gives $P^{(1)} = 0$. Unrolling Equation (2) from iteration 2 yields

$$P^{(t)} + \mathrm{vol}(R) \le \mathrm{vol}(R) \prod_{i=2}^{t} (1 + a_i).$$

If $\delta\theta > 1$, then $a_i = 0$ for every $i \ge 2$, proving $P^{(t)} = 0$. Suppose now that $0 < \delta\theta \le 1$, and let $N_{\delta,\theta} = \lfloor 2/(\delta\theta) \rfloor$. We have $a_i = 0$ for every $i > N_{\delta,\theta}$, while for $2 \le i \le N_{\delta,\theta}$,

$$1 + a_i = 1 + \frac{2 - \delta\theta i}{\delta\theta(i+2)} = \frac{2(1+\delta\theta)}{\delta\theta(i+2)}.$$

Consequently,

$$\begin{aligned}P^{(t)} &\le \left[\prod_{i=2}^{N_{\delta,\theta}} \frac{2(1+\delta\theta)}{\delta\theta(i+2)} - 1\right]\mathrm{vol}(R) \\ &= \left[\frac{6(2(1+\delta\theta)/\delta\theta)^{N_{\delta,\theta}-1}}{(N_{\delta,\theta}+2)!} - 1\right]\mathrm{vol}(R) \\ &= C_{\delta,\theta}\,\mathrm{vol}(R),\end{aligned}$$

which proves the lemma. □

B.3 Proof of Theorem 8

Proof. Substituting the bound $P^{(t)} \le C_{\delta,\theta}\,\mathrm{vol}(R)$ from Lemma 2 into Lemma 1 gives

$$\begin{aligned}\mathrm{vol}(A^{(t)}) &\le \left(1 + \frac{1}{\delta\theta}\right)\mathrm{vol}(R) + \frac{C_{\delta,\theta}}{\delta\theta}\,\mathrm{vol}(R) \\ &= \left(1 + \frac{1 + C_{\delta,\theta}}{\delta\theta}\right)\mathrm{vol}(R).\end{aligned}$$

□

B.4 Proof of Theorem 9

Proof. We work in the nontrivial parameter regime specified in Section 5.3. Every Dinkelbach value maintained by PCI-FW is the PCI objective value of a feasible set, so $\delta \ge 1/\mathrm{vol}(R)$. Define the δ-independent constant $\widehat{C} = C_{1/\mathrm{vol}(R),\theta}$. The product representation in the proof of Lemma 2 shows that $C_{\delta,\theta}$ is non-increasing in δ. Hence, $\delta \ge 1/\mathrm{vol}(R)$ implies $C_{\delta,\theta} \le \widehat{C}$. Theorem 8 therefore gives the uniform bound

$$\begin{aligned}\mathrm{vol}(A^{(t)}) &\le \left(1 + \frac{1 + C_{\delta,\theta}}{\delta\theta}\right)\mathrm{vol}(R) \\ &\le \left[1 + \frac{\mathrm{vol}(R)}{\theta}\big(1 + \widehat{C}\big)\right]\mathrm{vol}(R).\end{aligned} \tag{3}$$

Define $M_R(\theta) = \left[1 + \frac{\mathrm{vol}(R)}{\theta}(1 + \widehat{C})\right]\mathrm{vol}(R)$. This bound holds for every Frank–Wolfe iteration in every Dinkelbach round. Moreover,

$$|E_A^{(t)}| = |E(A^{(t)})| + |\partial E(A^{(t)})| \le \mathrm{vol}(A^{(t)}) \le M_R(\theta).$$

Thus, using adjacency lists and incrementally maintaining the net flows and residuals, one Frank–Wolfe iteration takes $O(M_R(\theta))$ time. A sweep check sorts the active vertices and scans their incident edges, taking $O(M_R(\theta)\log(1 + M_R(\theta)))$ time. Since T and T_{check} are fixed, the time and space used within one Dinkelbach round are both bounded by functions of $\mathrm{vol}(R)$ and the fixed parameter θ.

It remains to bound the number of Dinkelbach rounds. Initially, $\delta_0 = \phi_\theta(R; R) = \frac{\mathrm{cut}(R)}{\mathrm{vol}(R)}$. Every successful round finds a negative witness S and strictly decreases δ to $\phi_\theta(S; R)$. For such a set, let

$$a = \mathrm{cut}(S), \qquad b = \mathrm{vol}(S \cap R), \qquad c = \mathrm{vol}(S \setminus R).$$

These are nonnegative integers, and feasibility gives $b - \theta c > 0$. Since every accepted value is smaller than δ_0,

$$1 \le a < \delta_0(b - \theta c) \le \delta_0\,\mathrm{vol}(R) = \mathrm{cut}(R),$$

while $1 \le b \le \mathrm{vol}(R)$ and $0 \le c < \mathrm{vol}(R)/\theta$. The new Dinkelbach value is completely determined by the integer triple (a, b, c) through

$$\delta_{\mathrm{new}} = \frac{a}{b - \theta c}.$$

There are at most

$$\operatorname{cut}(R)\operatorname{vol}(R)\left\lceil\frac{\operatorname{vol}(R)}{\theta}\right\rceil$$

such triples. Because the accepted Dinkelbach values strictly decrease, the number of successful updates is no larger than this quantity; adding the final unsuccessful round changes the bound by at most one. Since $\operatorname{cut}(R) \le \operatorname{vol}(R)$ and θ is fixed, the total number of rounds is bounded solely in terms of $\operatorname{vol}(R)$. Combining this round bound with Equation (3) proves that both the total running time and the maximum space usage of PCI-FW are bounded in terms of $\operatorname{vol}(R)$, independently of $|V|$ and $|E|$. □

B.5 Proof of Theorem 10

Proof. We first verify that the active-edge update is a valid global Frank–Wolfe update. For each edge $e = (u, v)$, regard $q_e = f_{e,u} = -f_{e,v}$ as its independent flow variable. The derivative of the objective is

$$\frac{\partial H_\delta(f)}{\partial q_e} = r_v(f) - r_u(f).$$

On every edge in $E_A^{(t-1)}$, Algorithm 4 chooses the endpoint direction that minimizes this linearized objective. If $e \notin E_A^{(t-1)}$, then both endpoint residuals are zero and the derivative is zero, so a global Frank–Wolfe linear oracle may choose the current value $q_e^{(t-1)}$ and leave this edge unchanged. Therefore, the local direction used by PCI-FW, extended by the current flow on all inactive edges, is a valid global Frank–Wolfe direction.

We next obtain a local curvature bound. Let

$$M = \left(1 + \frac{1 + C_{\delta,\theta}}{\delta\theta}\right)\operatorname{vol}(R).$$

By Theorem 8, $|E_A^{(t)}| \le \operatorname{vol}(A^{(t)}) \le M$ at every iteration. Let B_t be the signed incidence matrix of $E_A^{(t-1)}$, with $B_{t,u,e} = 1$, $B_{t,v,e} = -1$, and all other entries zero for each edge $e = (u, v)$. The scalar function $x \mapsto \frac{1}{2}[x]_+^2$ is 1-smooth. Hence, for any feasible direction z supported on $E_A^{(t-1)}$ and any $\eta \in [0, 1]$,

$$H_\delta(f + \eta z) \le H_\delta(f) + \eta\langle \nabla H_\delta(f), z\rangle + \frac{\eta^2}{2}\|B_t z\|_2^2.$$

The standard Laplacian bound gives

$$\|B_t z\|_2^2 \le 2d_{\max}(E_A^{(t-1)})\|z\|_2^2 \le 2M\|z\|_2^2,$$

where $d_{\max}(E_A^{(t-1)}) = \max_{u\in V} |\{e \in E_A^{(t-1)} \mid e \ni u\}|$ denotes the maximum degree in the subgraph formed by the active edges. For a Frank–Wolfe direction, each active-edge coordinate changes by at most 2, and thus $\|z\|_2^2 \le 4|E_A^{(t-1)}| \le 4M$. It follows that the curvature along every update is uniformly bounded by

$$C_H \le 8M^2.$$

Let $h_t = H_\delta(f^{(t)}) - H_\delta^*$. The Dinkelbach value is initialized with $\delta_0 = \operatorname{cut}(R)/\operatorname{vol}(R) \le 1$ and only decreases, so $\delta \le 1$. Under the zero-flow initialization,

$$H_\delta(f^{(0)}) = \frac{1}{2}\delta^2 \sum_{u\in R} d_u^2 \le \frac{1}{2}\operatorname{vol}(R)^2 \le C_H.$$

The standard Frank–Wolfe curvature inequality, the global linear-oracle property proved above, and convexity imply [20]

$$h_t \le (1 - \gamma^{(t)})h_{t-1} + \frac{(\gamma^{(t)})^2}{2}C_H.$$

Substituting $\gamma^{(t)} = 2/(t + 2)$ and applying induction yields $h_t \le 2C_H/(t + 2)$. Using $C_H \le 8M^2$ therefore gives

$$0 \le H_\delta(f^{(t)}) - H_\delta^* \le \frac{16M^2}{t+2} = \frac{16\left(1 + (1 + C_{\delta,\theta})/(\delta\theta)\right)^2 \operatorname{vol}(R)^2}{t+2},$$

which proves the theorem. □